



Département Informatique

Ref : PFA2-2026-12

Rapport de Projet de Fin d'Année

de

Deuxième année en Génie Informatique

Présenté et soutenu publiquement le 13/05/2026

Par

**Faten OCHI
Adem RIAHI
Mohamed Bechir CHEMMAM**

Systeme intelligent de détection, prédiction et optimisation des embouteillages routiers

Composition du jury

Monsieur

Ramzi FARHAT

Président

Monsieur

Mehrez BOULARES

Encadrant

Année universitaire : 2025-2026

Dédicaces

À nos parents bien-aimés, piliers de notre vie, dont l'amour inconditionnel et les sacrifices consentis jour après jour ont nourri notre détermination et guidé chacun de nos pas.

À nos frères et sœurs, pour leur présence réconfortante, leur soutien sincère et les moments de complicité qui ont allégé les instants difficiles de ce parcours.

À nos amis, pour leur générosité, leur bonne humeur et leurs encouragements constants qui ont rendu ce travail plus léger et cette aventure plus belle.

Ce rapport est le fruit de vos sacrifices, de votre confiance et de votre amour. Chacun d'entre vous a contribué, à sa manière, à forger ce que nous sommes aujourd'hui. Cette réussite est aussi la vôtre.

Remerciements

Nous tenons à exprimer notre sincère gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet et à la rédaction de ce rapport.

Nous adressons nos plus vifs remerciements à **Monsieur Mehrez Boulares**, notre encadrant académique, pour son accompagnement rigoureux, sa disponibilité et la qualité de ses conseils tout au long de ce travail.

Nous remercions également **Monsieur Ramzi Farhat**, président du jury, pour l'intérêt qu'il a bien voulu porter à notre projet et pour son évaluation attentive.

Nos remerciements s'adressent enfin à l'ensemble du corps enseignant pour la rigueur et la richesse de la formation dispensée, ainsi qu'à toutes celles et ceux qui ont, par leur aide ou leur encouragement, contribué à l'aboutissement de ce travail.

Table des matières

Introduction générale	1
1 Cadre général du projet	3
Introduction	3
1 Présentation du projet	3
1.1 Problématique	3
1.2 Solutions existantes	4
1.2.1 État de l’art	4
1.2.2 Évaluation des solutions existantes et leurs limites	5
1.3 Description de notre solution	6
1.4 Objectifs du projet	7
2 Méthodologie CRISP-DM	7
2.1 Présentation de CRISP-DM	8
2.2 Application au projet	8
3 Compréhension métier	9
3.1 Définition du problème métier	9
3.2 Acteurs du système	10
3.3 Indicateurs de succès	10
4 Besoins fonctionnels	11
5 Besoins non fonctionnels	11
6 Environnement de développement	12
6.1 Présentation de SUMO	12
6.2 Environnement matériel	12
6.3 Environnement logiciel et bibliothèques	13
7 Architecture logique	13
Conclusion	14
2 Modélisation	15
Introduction	15

1	Apprentissage par Renforcement : Fondements Théoriques	15
1.1	Le Processus de Décision Markovien	15
1.2	Politiques et Retour Cumulé	16
1.3	Exploration et Exploitation	17
2	Configuration de l’Environnement d’Apprentissage	17
2.1	Paramètres réseau et temporels	17
2.2	Flux de trafic	18
2.2.1	Scénario 1 — Une intersection	19
2.2.2	Scénario 2 — Deux intersections	19
2.3	Paramètres de l’algorithme Q-Learning	20
2.4	Données extraites via TraCI	21
3	Diagramme d’Activité de l’Agent	22
4	Q-Learning	24
4.1	L’Équation d’Optimalité de Bellman	24
4.2	La Règle de Mise à Jour	24
4.3	Application au Contrôle des Feux	25
5	Fonction d’Activation	26
6	Deep Q-Network (DQN)	27
6.1	Mémoire de Relecture	27
6.2	Réseau Cible	28
6.3	Architecture du Réseau de Neurones	28
6.4	Procédure d’Entraînement	28
6.5	Extension Multi-Intersections	30
6.5.1	Independent Q-Learning (IQL)	30
6.5.2	Deep Q-Learning Partagé	30
7	Analyse Comparative : Q-Learning et DQN	31
8	Baseline : Feux à Cycles Fixes	32
	Conclusion	32
3	Évaluation	33
	Introduction	33
1	Scénarios de simulation	33
1.1	Scénario 1 : Une seule intersection	33

1.2	Scénario 2 : Deux intersections interconnectées	34
1.3	Conditions communes aux deux scénarios	35
1.4	Objectif des scénarios	35
2	Métriques d'évaluation	35
2.1	Temps d'attente moyen	36
2.2	Longueur moyenne des files d'attente	36
2.3	Débit de véhicules	36
2.4	Fonction de récompense	36
2.4.1	Formulation du problème multi-objectif	37
2.4.2	Transformation en objectif scalaire	37
2.4.3	Problème de déséquilibre des échelles	37
2.4.4	Normalisation par pondération	38
3	Résultats et analyse	38
3.1	Résultats — 1 intersection	38
3.1.1	Courbes d'entraînement du Q-Learning	38
3.1.2	Courbes d'entraînement du DQN	40
3.1.3	Résultats comparatifs — 1 intersection	42
3.1.4	Conclusion — 1 intersection	43
3.2	Résultats — 2 intersections	43
3.2.1	Courbes d'entraînement du Q-Learning — 2 intersections	43
3.2.2	Courbes d'entraînement du DQN — 2 intersections . . .	45
3.2.3	Feux fixes — 2 intersections	47
3.2.4	Résultats comparatifs — 2 intersections	48
3.2.5	Conclusion — 2 intersections	48
3.3	Comparaison globale	48
4	Discussion	49
4.1	Interprétation des résultats	50
4.2	Limites observées	50
	Conclusion	50

Conclusion générale et perspectives 51

Table des figures

1.1	Les six phases de la méthodologie CRISP-DM	8
1.2	Architecture logique du système	14
2.1	Diagramme d'activité de l'agent RL	23
2.2	Organigramme algorithmique du Q-Learning appliqué au contrôle des feux	26
2.3	Fonction d'activation ReLU : $f(x) = \max(0, x)$	27
2.4	Architecture du réseau de neurones du DQN pour le contrôle des feux sur une intersection unique	28
2.5	Boucle d'entraînement du DQN avec mémoire de rejeu et réseau cible dans l'environnement de trafic SUMO	29
2.6	Architecture du réseau de neurones du DQN pour le contrôle des feux sur deux intersections	31
3.1	Vue SUMO de l'intersection unique à quatre branches (scénario 1)	34
3.2	Vue SUMO du réseau à deux intersections interconnectées (scénario 2)	35
3.3	Q-Learning Récompense cumulée par épisode (asymptote : $-77,47$)	39
3.4	Q-Learning Temps d'attente moyen par épisode (asymptote : $13,69$ s)	39
3.5	Q-Learning Longueur moyenne de file par épisode (asymptote : $1,81$ véhicule)	40
3.6	Q-Learning Flux moyen par épisode (asymptote : $1,89$ véhicule/pas)	40
3.7	DQN Récompense cumulée par épisode (asymptote : $-61,18$)	41
3.8	DQN Temps d'attente moyen par épisode (asymptote : $11,75$ s)	41
3.9	DQN Longueur moyenne de file par épisode (asymptote : $1,75$ véhicule) . .	42
3.10	DQN Flux moyen par épisode (asymptote : $1,90$ véhicule/pas)	42
3.11	Q-Learning (2 intersections) Récompense cumulée par épisode	44
3.12	Q-Learning (2 intersections) Temps d'attente moyen par épisode	44
3.13	Q-Learning (2 intersections) Longueur moyenne de file par épisode	45
3.14	Q-Learning (2 intersections) Flux moyen par épisode	45
3.15	DQN (2 intersections) Récompense cumulée par épisode	46
3.16	DQN (2 intersections) Temps d'attente moyen par épisode	46

3.17 DQN (2 intersections) Longueur moyenne de file par épisode	47
3.18 DQN (2 intersections) Flux moyen par épisode	47

Liste des tableaux

1.1	Tableau comparatif des approches de gestion du trafic	5
1.2	Environnement matériel	12
1.3	Environnement logiciel et bibliothèques	13
2.1	Paramètres réseau et temporels de la simulation SUMO — comparaison 1 intersection / 2 intersections	18
2.2	Flux de trafic — 1 intersection (probabilité d'arrivée/seconde)	19
2.3	Flux de trafic — 2 intersections (probabilité d'arrivée/seconde)	20
2.4	Paramètres de l'algorithme Q-Learning et justification des choix	21
2.5	Données extraites via TraCI et leur utilisation dans l'apprentissage	22
2.6	Comparaison Q-Learning et Deep Q-Network dans le contexte du projet . .	31
2.7	Feux fixes : comparaison	32
3.1	Résultats comparatifs des trois approches — 1 intersection	43
3.2	Résultats comparatifs des trois approches — 2 intersections	48
3.3	Comparaison globale des trois approches sur les deux scénarios	49

Introduction générale

La congestion du trafic urbain représente l'un des défis les plus pressants de notre époque. Dans les métropoles du monde entier, les embouteillages engendrent chaque année des pertes économiques colossales, aggravent la pollution atmosphérique et dégradent la qualité de vie de millions de citoyens. Face à cette réalité, les systèmes de gestion du trafic traditionnels qu'il s'agisse des feux à cycles fixes, des agents de circulation humains ou des applications de navigation montrent leurs limites : figés dans des règles prédéfinies, ils sont incapables de s'adapter à la variabilité et à l'imprévisibilité des flux réels.

L'intelligence artificielle, et plus particulièrement l'apprentissage par renforcement, ouvre une voie prometteuse pour surmonter ces limitations. Contrairement aux approches classiques, un agent par renforcement n'a besoin d'aucun jeu de données préexistant : il apprend exclusivement par interaction directe avec son environnement, en explorant différentes décisions, en observant leurs conséquences et en ajustant progressivement son comportement pour maximiser une récompense cumulée. Cette capacité d'apprentissage autonome et d'adaptation continue en fait un paradigme particulièrement adapté au contrôle dynamique des carrefours.

C'est dans ce contexte que s'inscrit le présent projet, dont l'objectif est de concevoir, d'implémenter et d'évaluer un système intelligent de gestion des feux de signalisation fondé sur l'apprentissage par renforcement. Pour valider l'approche sans risque sur une infrastructure réelle, les expérimentations sont menées dans SUMO (Simulation of Urban MObility), un simulateur de trafic open-source reconnu par la communauté scientifique. Deux algorithmes sont comparés : le Q-Learning, dont la simplicité et les garanties théoriques de convergence en font un point de départ naturel, et le Deep Q-Network (DQN), qui étend ces capacités aux espaces d'états continus et aux configurations multi-intersections grâce à l'approximation neuronale.

Ce rapport est structuré en trois chapitres complémentaires. Le **premier chapitre** présente le cadre général du projet : problématique, état de l'art, description de la solution proposée, méthodologie CRISP-DM adoptée, compréhension métier et environnement de développement. Le **deuxième chapitre** développe le cadre de modélisation, depuis la formalisation du problème sous forme de Processus de Décision Markovien jusqu'à l'im-

plémentation détaillée des algorithmes Q-Learning et DQN, en passant par la configuration précise de l'environnement de simulation. Le **troisième chapitre** présente l'évaluation expérimentale sur deux scénarios de complexité croissante une intersection unique puis un réseau de deux intersections interconnectées et analyse les résultats obtenus au regard des objectifs fixés.

Chapitre 1

Cadre général du projet

Introduction

Ce chapitre présente le contexte et les bases du projet. Après une analyse de la problématique et des solutions existantes, nous décrivons notre solution et ses objectifs, puis exposons la méthodologie adoptée, le cadre métier et l’environnement de développement.

1 Présentation du projet

Cette section introduit le contexte général du projet, en partant du problème de la congestion urbaine pour aboutir à la description de notre solution et de ses objectifs. L’analyse des approches existantes permet de justifier le recours à l’apprentissage par renforcement comme paradigme central.

1.1 Problématique

La congestion du trafic urbain constitue l’un des défis les plus coûteux et les plus pressants auxquels font face les villes du monde entier. Ses conséquences dépassent le simple inconvénient quotidien pour toucher des dimensions économiques, environnementales et sanitaires d’une ampleur considérable. Sur le plan économique, les conducteurs de grandes métropoles comme Londres, Paris et Chicago ont perdu en moyenne entre 96 et 101 heures dans les embouteillages en 2023, et en 2024 la congestion a coûté au Royaume-Uni plus de 7,7 milliards de livres sterling, soit environ 942 livres perdues par conducteur londonien en temps et en carburant gaspillés [12]. Sur le plan environnemental, le secteur des transports a émis 8,4 gigatonnes de CO₂ en 2024, représentant près de 16% des émissions mondiales de gaz à effet de serre ; ce bilan est aggravé par la congestion elle-même, puisqu’un véhicule à l’arrêt consomme sensiblement plus qu’un véhicule en régime stabilisé [11]. Sur le plan sanitaire, la pollution atmosphérique liée au trafic a été associée à environ 4,2 millions de décès prématurés dans le monde en 2019, avec un impact particulièrement sévère dans les

pays en développement où 98% des villes ne respectent pas les normes de qualité de l'air de l'OMS [10]. Face à ce constat, les solutions actuelles que sont les feux à cycles fixes, les agents humains et les applications de navigation se révèlent insuffisantes et incapables de s'adapter à la variabilité des flux réels, ce qui justifie la recherche d'une approche plus intelligente et autonome.

1.2 Solutions existantes

Face à la persistance du problème de congestion, de nombreuses approches ont été proposées au fil des décennies. Cette sous-section en dresse un panorama critique, depuis les premières tentatives adaptatives jusqu'aux méthodes les plus récentes intégrant l'intelligence artificielle.

1.2.1 État de l'art

La gestion du trafic aux carrefours a évolué des systèmes purement mécaniques vers des approches intégrant l'intelligence artificielle.

Les **systèmes adaptatifs à base de capteurs** constituent les premières tentatives sérieuses. SCATS (Sydney Coordinated Adaptive Traffic System), déployé dès les années 1980 en Australie et adopté dans plus de 170 villes, ajuste les cycles des feux toutes les 90 secondes à partir de données de boucles magnétiques [16]. SCOOT (Split Cycle Offset Optimisation Technique), utilisé à Londres et Singapour, optimise les décalages entre carrefours voisins pour créer des ondes vertes [17].

Les **approches par logique floue** ont introduit l'intelligence artificielle dans ce domaine. Pappis et Mamdani [18] ont défini des règles linguistiques imitant un opérateur humain expérimenté, tandis que Trabia et al. [19] ont enrichi cette approche avec un contrôleur à deux niveaux gérant simultanément la durée des phases et la coordination entre carrefours.

Les **algorithmes évolutionnaires** ont permis d'optimiser les plans de phases. Sanchez et al. [20] ont utilisé des algorithmes génétiques sur un réseau de carrefours interconnectés, et Teklu et al. [21] les ont combinés avec des modèles de simulation pour des configurations à l'échelle urbaine.

Les **approches d'apprentissage supervisé** ont visé la prédiction des flux. Zhang et al. [22] ont proposé un modèle LSTM-convolutif atteignant une précision supérieure à 90% sur des données de Beijing. Lv et al. [23] ont utilisé des auto-encodeurs profonds pour

anticiper les volumes de circulation.

Les **systèmes de transport intelligents** exploitent la communication véhicule-infrastructure. Le projet COMPASS4D en Europe et le déploiement de Columbus (Ohio) en 2017 ont démontré des réductions de 25% des temps d'intervention des véhicules d'urgence [25].

Les **approches par vision par ordinateur** utilisent les caméras urbaines existantes. Lim et al. [24] ont appliqué des réseaux convolutifs pour compter les véhicules en temps réel, sans capteurs enfouis dans la chaussée.

Le tableau 1.1 présente une synthèse comparative de ces approches selon les critères pertinents pour notre projet.

TABLE 1.1 – *Tableau comparatif des approches de gestion du trafic*

Approche	Adaptation temps réel	Apprentissage	Scalabilité	Coût infrastructure
Feux fixes	Non	Non	Élevée	Faible
SCATS / SCOOT	Partielle	Non	Moyenne	Élevé
Logique floue	Non	Non	Faible	Moyen
Alg. génétiques	Non (hors ligne)	Non	Moyenne	Faible
Apprentissage supervisé	Non	Oui (passif)	Élevée	Moyen
Vision par ordinateur	Partielle	Non	Moyenne	Moyen
Véhicules connectés	Oui	Non	Faible	Très élevé

1.2.2 Évaluation des solutions existantes et leurs limites

Les solutions présentées partagent des limitations structurelles qui réduisent leur efficacité face à la variabilité des conditions réelles. Cette analyse justifie le recours à l'apprentissage par renforcement.

L'agent de circulation humain, malgré sa réactivité, est limité par la fatigue, le champ de vision restreint et l'impossibilité d'être présent simultanément sur plusieurs carrefours. Sa présence est coûteuse et non scalable à l'échelle d'un réseau urbain [10].

Les feux à cycles fixes sont fondamentalement incapables de s'adapter à la réalité dynamique : un feu configuré pour un mardi ordinaire applique le même cycle lors d'un événement sportif ou d'un accident, aggravant la congestion au lieu de la résorber [13].

Les applications de navigation telles que Google Maps et Waze redistribuent simplement le trafic vers d'autres axes, créant de nouveaux embouteillages sur des routes secondaires. Lorsque tous les conducteurs suivent la même déviation, l'itinéraire alternatif devient à son tour saturé [14].

Les ronds-points peuvent se bloquer totalement en heure de pointe lorsque le flux entrant dépasse leur capacité, générant des engorgements circulaires parfois plus critiques qu'un carrefour classique [15].

SCATS et SCOOT souffrent d'une dépendance à une infrastructure physique lourde, d'un temps de réaction minimum de 90 secondes et d'une incapacité à gérer les perturbations imprévues [16, 17].

La logique floue et les algorithmes génétiques requièrent une expertise humaine pour définir les règles et opèrent en mode hors ligne, ce qui les empêche de s'adapter dynamiquement une fois déployés [18, 20].

L'apprentissage supervisé dépend des données historiques et reste inefficace face à des situations inédites. Ces méthodes prédisent mais ne décident pas, et ne peuvent générer de nouvelles stratégies de contrôle [23].

En somme, aucune de ces solutions n'est capable d'apprendre de manière autonome, de s'améliorer avec l'expérience et de décider optimalement face à des situations nouvelles, ce qui motive le recours à l'apprentissage par renforcement.

1.3 Description de notre solution

Notre solution propose un système intelligent de gestion du trafic fondé sur l'apprentissage par renforcement, capable de contrôler dynamiquement les feux de signalisation sur un réseau de routes et carrefours interconnectés. Un agent intelligent observe en continu l'état du trafic et prend en temps réel les meilleures décisions sur la durée des phases de feux afin de minimiser les temps d'attente et maximiser la fluidité. Contrairement aux systèmes classiques qui s'appuient sur des règles figées ou des données préexistantes, l'agent n'a besoin d'aucun jeu de données initial : il apprend exclusivement par interaction directe avec son environnement, en explorant différentes décisions, en observant leurs conséquences sur

le trafic et en ajustant progressivement son comportement pour maximiser sa récompense. Il s'améliore ainsi à chaque itération et s'adapte naturellement à toute situation nouvelle. Pour valider cette approche sans risque sur une infrastructure réelle, nous utilisons SUMO (Simulation of Urban MObility), un simulateur de trafic open-source reconnu dans la communauté scientifique [28].

1.4 Objectifs du projet

Notre projet s'articule autour de six objectifs complémentaires, définis de façon à être mesurables à partir des métriques produites par le simulateur SUMO.

- Modéliser l'environnement de trafic comme un espace d'apprentissage dans lequel un agent peut observer, décider et apprendre.
- Réduire les temps d'attente des véhicules aux feux en optimisant dynamiquement la durée des phases selon l'état réel du trafic.
- Gérer les pics de trafic soudains en adaptant automatiquement le comportement du système sans intervention humaine.
- Entraîner et comparer le Q-Learning et le Deep Q-Learning pour identifier l'approche la plus performante.
- Valider la solution par simulation sur des scénarios de trafic réalistes avec SUMO.
- Produire des résultats exploitables sous forme de métriques claires : temps d'attente moyen, longueur des files et taux de fluidification.
- Évaluer la **scalabilité** de l'approche en étendant le contrôle d'une intersection unique à un réseau de deux intersections interconnectées, afin de vérifier que les gains en performance se maintiennent lorsque la complexité de l'environnement augmente.

2 Méthodologie CRISP-DM

La rigueur d'un projet d'intelligence artificielle repose en grande partie sur la méthodologie qui en structure le déroulement. Cette section présente le cadre CRISP-DM retenu pour ce projet et détaille son application concrète à chacune des étapes du développement.

2.1 Présentation de CRISP-DM

CRISP-DM (Cross-Industry Standard Process for Data Mining) est une méthodologie de référence adoptée dans les projets d'intelligence artificielle et de science des données [26]. Elle structure le déroulement en six phases itératives et interconnectées, permettant une approche organisée et orientée résultats. CRISP-DM est particulièrement adaptée aux projets où les données et les modèles d'apprentissage sont au cœur du processus, ce qui en fait un cadre naturel pour notre travail.



FIGURE 1.1 – *Les six phases de la méthodologie CRISP-DM*

Ces six phases sont la compréhension métier, la compréhension des données, la préparation des données, la modélisation, l'évaluation et le déploiement. Elles ne sont pas strictement séquentielles : la méthodologie recommande explicitement de revenir à une phase antérieure lorsque les résultats d'une phase ultérieure le nécessitent, ce qui lui confère une grande flexibilité [26].

2.2 Application au projet

Dans notre projet, les six phases de CRISP-DM correspondent aux étapes concrètes du développement. Il convient de noter que la **Phase 6 — Déploiement** reste à ce stade une perspective future : dans le périmètre actuel du projet, le système n'a pas fait l'objet d'un déploiement opérationnel sur une infrastructure réelle ; les développements réalisés se

limitent aux phases 1 à 5, la phase 6 constituant un axe d’extension identifié pour de futurs travaux.

- **Phase 1 Compréhension métier** : définition du problème de congestion, identification des acteurs et fixation des indicateurs de succès (voir section 3).
- **Phase 2 Compréhension des données** : exploration des données de trafic générées par SUMO, incluant densité des véhicules, longueur des files et phase des feux (voir section 2.1).
- **Phase 3 Préparation des données** : mise en forme pour les algorithmes RL, définition de l’espace d’états, d’actions et de la fonction de récompense (voir section 2.1).
- **Phase 4 Modélisation** : implémentation et entraînement du Q-Learning et du Deep Q-Learning dans l’environnement SUMO (voir sections 2.4 et 2.6).
- **Phase 5 Évaluation** : test des modèles sur des scénarios variés et mesure des performances (voir Chapitre 3).
- **Phase 6 Déploiement** (*perspective future*) : intégration des résultats dans une interface web et mobile destinée aux policiers routiers. Cette phase n’a pas été développée dans le cadre du présent projet et constitue un axe d’extension pour de futurs travaux.

3 Compréhension métier

La compréhension du contexte métier constitue la première étape indispensable avant toute modélisation algorithmique. Cette section identifie les acteurs concernés, définit précisément le problème à résoudre et établit les indicateurs qui permettront de mesurer le succès du système. Elle correspond à la **Phase 1** de la méthodologie CRISP-DM présentée à la section 1.2.

3.1 Définition du problème métier

Le problème de la congestion urbaine impacte directement plusieurs acteurs au quotidien. Les automobilistes sont les premiers touchés, subissant des pertes de temps considérables lors des heures de pointe, avec des conséquences directes sur leur productivité et leur qualité de vie. Les policiers routiers se retrouvent souvent dépassés face à des situations évoluant rapidement sur plusieurs points du réseau simultanément, sans outils leur permettant

d’anticiper ou de réagir efficacement. L’absence d’un système intelligent se traduit par une accumulation des files d’attente, une dégradation continue du service et une incapacité totale à gérer les imprévus, ce qui constitue le problème métier central que notre projet cherche à résoudre.

3.2 Acteurs du système

Notre système distingue quatre acteurs aux rôles distincts.

- **Le policier routier** : acteur direct et utilisateur principal. Il consulte l’état du trafic, reçoit des alertes sur les zones congestionnées et s’appuie sur les recommandations pour décider d’intervenir.
- **Le gestionnaire du trafic** : acteur de supervision. Il exploite les statistiques produites par le système pour orienter les décisions stratégiques.
- **L’automobiliste** : acteur indirect et bénéficiaire. Il n’interagit pas avec le système mais profite de ses effets à travers la réduction des temps d’attente.
- **L’agent IA** : acteur central et cœur intelligent du système. Il observe le trafic, apprend par renforcement et prend automatiquement les meilleures décisions sur les phases des feux.

3.3 Indicateurs de succès

Les indicateurs de succès constituent les critères objectifs permettant de mesurer l’efficacité réelle du système par rapport à une baseline de référence. Ils ont été définis de manière à être directement mesurables dans l’environnement de simulation SUMO.

- **Temps d’attente moyen** : durée moyenne qu’un véhicule passe à l’arrêt avant de traverser un carrefour. Il s’agit de l’indicateur principal de succès.
- **Longueur des files d’attente** : nombre moyen de véhicules en attente par voie à chaque point de passage.
- **Débit véhiculaire** : nombre de véhicules traversant un carrefour par unité de temps.
- **Taux d’optimisation** : gain apporté par notre système par rapport aux feux fixes, exprimé en pourcentage.
- **Temps de convergence** : nombre d’épisodes d’entraînement nécessaires pour que l’agent atteigne une politique stable et performante. Cet indicateur permet d’évaluer

l'efficacité algorithmique des approches comparées (Q-Learning vs DQN) et conditionne leur applicabilité pratique dans des contextes où la durée d'apprentissage est contrainte.

4 Besoins fonctionnels

Le système doit répondre à un ensemble de besoins fonctionnels précis qui définissent ses capacités opérationnelles. Ces besoins ont été identifiés à partir de l'analyse du contexte métier et des objectifs fixés pour l'agent d'apprentissage.

- **Simulation du trafic** : simuler le comportement des véhicules sur un réseau de routes et ronds-points via SUMO.
- **Détection de la congestion** : détecter automatiquement les zones congestionnées en analysant la densité des véhicules, les files et les temps d'attente.
- **Optimisation des feux** : ajuster dynamiquement la durée des phases afin de minimiser les temps d'attente et maximiser le débit.
- **Apprentissage automatique** : entraîner un agent Q-Learning et Deep Q-Learning pour qu'il apprenne de manière autonome la meilleure politique de gestion.
- **Visualisation des résultats** : afficher les résultats sous forme de métriques claires.

5 Besoins non fonctionnels

Au-delà des fonctionnalités opérationnelles, le système doit également satisfaire des contraintes non fonctionnelles qui conditionnent sa qualité, sa robustesse et sa viabilité à long terme dans un contexte de déploiement réel.

- **Précision** : produire des décisions générant une réduction mesurable des temps d'attente.
- **Adaptabilité** : s'adapter à différentes configurations de réseaux sans reconfiguration manuelle.
- **Maintenabilité** : code structuré, documenté et modulaire pour faciliter les évolutions futures.

6 Environnement de développement

Cette section décrit l’environnement de simulation retenu ainsi que les configurations matérielles et logicielles dans lesquels le projet a été développé. La connaissance de cet environnement est nécessaire pour assurer la reproductibilité des résultats et évaluer les conditions d’exécution des simulations.

6.1 Présentation de SUMO

SUMO (Simulation of Urban MObility) est un simulateur de trafic open-source, microscopique et portable. Son développement a débuté en 2001 au sein du Centre Aérospatial Allemand (DLR) et il est devenu en 2018 un projet officiel de la Fondation Eclipse [28]. SUMO est purement microscopique : chaque véhicule est modélisé explicitement avec son propre itinéraire et se déplace individuellement selon le modèle de poursuite de Krauss [31].

Dans notre projet, SUMO constitue l’environnement d’apprentissage pour plusieurs raisons. Il permet de modifier les phases des feux en temps réel, de simuler un réseau sans limite de taille, et est disponible sur les principaux systèmes d’exploitation [29]. Il fournit TraCI, qui permet à l’agent Python d’interagir en temps réel avec la simulation [27]. Des expériences menées avec un agent DQN sur SUMO ont démontré des réductions des temps d’attente de 35 à 40% [8], confirmant sa pertinence pour valider notre approche.

6.2 Environnement matériel

Les simulations et le développement ont été réalisés sur les postes personnels des membres de l’équipe. Le tableau 1.2 présente les configurations utilisées.

TABLE 1.2 – *Environnement matériel*

Composant	Faten Ochi	Mohamed Bechir Chemmam	Adem Riahi
Processeur	i5-13500H	i7-12650H	i7-10870H
RAM	24,0 Go	24,0 Go	16,0 Go
Disque dur	500 Go	500 Go	477 Go
Système d’exploitation	Windows 11	Windows 10	Windows 11

6.3 Environnement logiciel et bibliothèques

Python, combiné à l'interface TraCI de SUMO, forme la boucle de contrôle centrale entre l'agent et l'environnement [27]. Le tableau 1.3 présente l'ensemble des outils et bibliothèques utilisés, en précisant le rôle de chacun dans l'architecture du projet.

TABLE 1.3 – *Environnement logiciel et bibliothèques*

Outil / Biblio- thèque	Rôle
SUMO	Simulateur de trafic urbain open-source constituant l'environnement de simulation
Python 3.x	Langage de programmation principal pour l'implémentation des algorithmes
TraCI	Interface de communication entre Python et SUMO pour le contrôle des feux en temps réel
NumPy	Calcul numérique et manipulation des tableaux
Matplotlib	Visualisation des courbes de récompenses et des résultats
Pickle	Sauvegarde et chargement des Q-tables et des données d'entraînement
Random	Génération d'actions aléatoires dans la politique d'exploration
OS	Gestion des chemins de fichiers et vérification des sauvegardes
Time	Délais entre les pas de simulation lors de la visualisation
Visual Studio Code	IDE pour l'édition, l'exécution et le débogage du code
Draw.io	Conception du diagramme d'activité de l'agent
Overleaf	Éditeur LaTeX pour la rédaction du présent rapport

7 Architecture logique

L'architecture logique du système décrit la manière dont ses composants s'organisent et interagissent pour former un pipeline cohérent allant de la simulation du trafic jusqu'aux décisions de l'agent d'apprentissage. Le système est structuré en trois couches interdépendantes, illustrées par la figure 1.2.

— **Couche simulation** : assurée par SUMO, elle génère en continu les données brutes

du trafic (position, vitesse, files d'attente, temps d'attente).

- **Couche communication** : assurée par TraCI, elle établit un pont bidirectionnel entre SUMO et le code Python, permettant l'extraction des données et l'envoi des commandes sur les feux [27].
- **Couche agent** : cœur du système, elle héberge l'agent RL qui observe, décide, reçoit la récompense et met à jour sa connaissance via Q-Learning ou DQN.

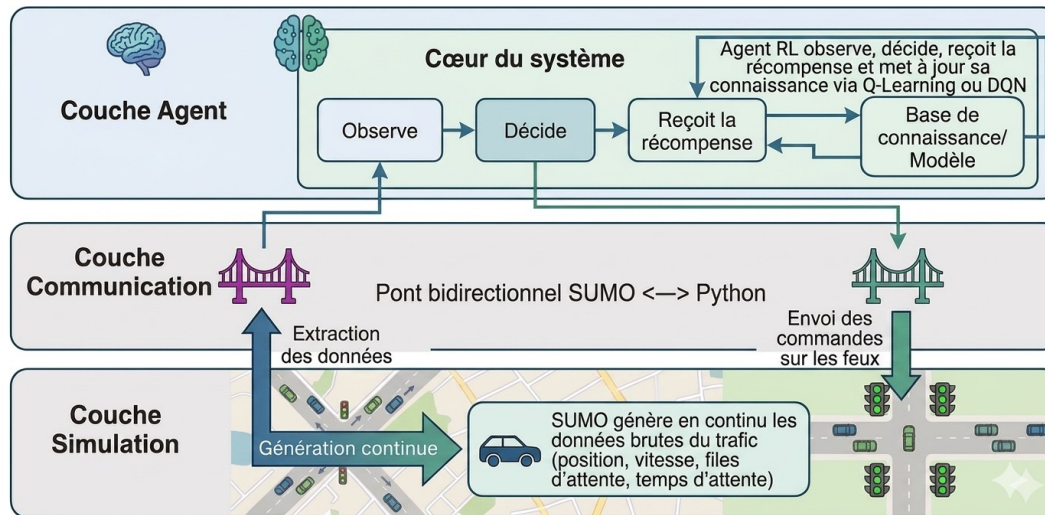


FIGURE 1.2 – Architecture logique du système

Conclusion

Ce chapitre a posé l'ensemble des bases conceptuelles et organisationnelles nécessaires au développement du projet. Partant de la problématique de la congestion urbaine et des limites des solutions existantes, nous avons justifié le recours à l'apprentissage par renforcement, précisé les besoins fonctionnels et non fonctionnels du système, présenté la méthodologie CRISP-DM adoptée en soulignant que la phase de déploiement (Phase 6) constitue une perspective future hors périmètre du présent travail et décrit l'environnement de développement retenu, incluant la présentation du simulateur SUMO. Le chapitre suivant se consacre à la modélisation formelle du problème, au paramétrage détaillé de l'environnement de simulation et à l'implémentation des algorithmes de Reinforcement Learning.

Chapitre 2

Modélisation

Introduction

Ce chapitre développe le cadre complet de modélisation du projet. Il s'articule autour de quatre axes principaux : la formalisation théorique du problème sous forme de Processus de Décision Markovien, la configuration précise de l'environnement de simulation et des données d'apprentissage, l'illustration du comportement de l'agent par un diagramme d'activité, et enfin la présentation et la comparaison des deux algorithmes retenus Q-Learning et Deep Q-Network. Cette organisation garantit une progression logique depuis les fondements mathématiques jusqu'aux détails d'implémentation.

1 Apprentissage par Renforcement : Fondements Théoriques

L'apprentissage par renforcement (RL) constitue le paradigme central sur lequel repose l'ensemble de notre solution. Cette section en présente les bases théoriques indispensables : la formalisation du problème en Processus de Décision Markovien, la définition des politiques et du retour cumulé, ainsi que la stratégie d'équilibre entre exploration et exploitation.

L'apprentissage par renforcement s'intéresse au problème d'un agent qui apprend une politique de sélection d'actions afin de maximiser un signal de récompense cumulé [1]. Ce qui le distingue des autres paradigmes d'apprentissage est l'absence d'exemples étiquetés et la dépendance à un retour évaluatif différé. L'agent explore les conséquences de ses propres actions pour estimer leur valeur à long terme. Dans notre projet, l'agent interagit avec SUMO via TraCI, qui fournit les observations nécessaires et exécute les décisions de contrôle.

1.1 Le Processus de Décision Markovien

La formalisation standard d'un problème de RL est le Processus de Décision Markovien (PDM), introduit par Bellman [2] et adopté par Sutton et Barto [1]. Un PDM est un

tuplet $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ dont chaque élément possède une signification précise.

- **Espace d'états** $\mathcal{S}$: l'état représente la situation du trafic à un instant donné, limité au nombre de véhicules arrêtés sur les deux axes. Pour le Q-Learning, il est discrétisé en (ns_bin, ew_bin) via les BINS = [0,3,6,9,12]. Pour le DQN, il est continu et normalisé : $[ns/10.0, ew/10.0]$.
- **Espace d'actions** $\mathcal{A}$: l'agent choisit entre deux phases à chaque pas de décision : phase Nord-Sud (NS) avec Est-Ouest au rouge, ou phase Est-Ouest (EW) avec Nord-Sud au rouge.
- **Fonction de transition** $P(s'|s, a)$: implicitement définie par la dynamique de SUMO, inconnue de l'agent, ce qui motive le recours aux méthodes sans modèle (*model-free*) [1].
- **Fonction de récompense** $R(s, a)$: signal scalaire évaluant la qualité de l'action, combinant trois composantes agrégées sur 10 secondes :

$$R = -w_{waiting} \cdot waiting - w_{queue} \cdot queue + w_{flow} \cdot flow \quad (2.1)$$

Les poids $w_{waiting} = w_{queue} = 1$ pénalisent la congestion et $w_{flow} = 1$ encourage le débit [1, 8, 9]. Ce type de fonction de récompense composite est courant dans la littérature sur le contrôle des feux par RL [8, 9]. L'agrégation sur le pas de temps lisse les variations instantanées et fournit un signal d'apprentissage plus stable.

- **Facteur d'actualisation** $\gamma \in [0, 1)$: contrôle l'importance des récompenses futures [1]. Une valeur proche de 1 ($\gamma = 0.95$) incite l'agent à adopter une vision à long terme.

La propriété de Markov stipule que l'évolution future ne dépend que de l'état courant [2] :

$$P(s_{t+1}|s_t, a_t, s_{t-1}, \dots) = P(s_{t+1}|s_t, a_t) \quad (2.2)$$

1.2 Politiques et Retour Cumulé

Une politique π associe des états à des distributions sur les actions [1]. L'objectif est de trouver la politique optimale π^* maximisant le retour actualisé :

$$G_t = \sum_{k=0}^{\infty} \gamma^k \cdot r_{t+k+1} \quad (2.3)$$

Maximiser ce retour (2.3) revient à trouver la stratégie de commutation minimisant les temps d'attente et les files tout en maximisant le débit sur l'horizon de simulation.

1.3 Exploration et Exploitation

Tout agent RL doit arbitrer entre **exploration** tester de nouvelles actions et **exploitation** s'appuyer sur ce qu'il a déjà appris. Nous utilisons la politique ε -greedy [1] : avec une probabilité ε l'agent agit aléatoirement, avec $1 - \varepsilon$ il choisit l'action de valeur Q maximale. ε démarre à 1.0 et décroît par un facteur 0.999 jusqu'à 0.05, ce qui pousse l'agent à explorer en début d'entraînement puis à exploiter ses acquis au fur et à mesure.

2 Configuration de l'Environnement d'Apprentissage

La mise en œuvre pratique du cadre théorique défini à la section 2.1 nécessite une configuration précise de l'environnement de simulation. Cette section détaille l'ensemble des paramètres qui régissent le réseau routier, les flux de trafic, les hyperparamètres de l'algorithme Q-Learning ainsi que les données extraites par TraCI à chaque pas de décision. Ces éléments constituent le substrat empirique sur lequel repose l'entraînement des agents et garantissent la reproductibilité des expériences.

2.1 Paramètres réseau et temporels

Le réseau simulé et les paramètres temporels de la simulation définissent le cadre physique dans lequel l'agent évolue et prend ses décisions. Le tableau 2.1 détaille l'ensemble des paramètres configurés dans SUMO et le code Python, en distinguant les valeurs propres au scénario mono-intersection (1 carrefour) de celles adoptées pour le scénario bi-intersection (2 carrefours).

TABLE 2.1 – Paramètres réseau et temporels de la simulation SUMO — comparaison 1 intersection / 2 intersections

Paramètre	1 intersection	2 intersections
Durée de simulation	0 s $\rightarrow$ 3 600 s (1 h/épisode)	0 s $\rightarrow$ 3 600 s (1 h/épisode)
Nombre d'épisodes	5 000	5 000
Topologie du réseau	1 carrefour à 4 bras (J1)	2 carrefours à 4 bras : J1 à l'origine (0,0) et J5 à 200 m, reliés par le tronçon E4/-E4 ($\approx$ 92,8 m)
Edges (arêtes)	E0-E3 et inverses; longueur $\approx$ 93 m, $v_{\max} \approx$ 50 km/h	J1 : E0-E3 et inverses (NS : E2_0, E3_0; EW : E1_0, E0_0). J5 : E4-E7 et inverses (NS : E5_0, E7_0; EW : E6_0, -E4_0)
Type de véhicule	Long. 5 m, accél. 2,6 m/s ² , décél. 4,5 m/s ² , $v_{\max} =$ 13,89 m/s	Identique; routes_2.rou.xml définit maxSpeed="10" m/s
Feux de signalisation	J1 : vert NS 42 s — vert EW 42 s — jaune 3 s	J1 et J5 : phase 0 vert NS 42 s; phase 1 jaune 3 s; phase 2 vert EW 42 s; phase 3 jaune 3 s
Pas de décision	10 s (T_DECISION = 10)	10 s (T_DECISION = 10)
Durée min. phase verte	5 s	5 s
Steps max / épisode	600 (MAX_STEPS = 600)	600 (MAX_STEPS = 600); horizon effectif $\approx$ 600 s
Discrétisation des états	[0, 3, 6, 9, 12] véhicules arrêtés	[0, 3, 6, 9, 12] véhicules arrêtés (BINS dans IQL_training.py)
Espace d'états	Discret : (ns_{bin}, ew_{bin}) (Q-Learning); continu normalisé : [$ns/10, 0, ew/10, 0$] (DQN)	Discret par intersection (IQL); continu normalisé partagé (DQL partagé)
Fonction de récompense	$R = -w_w \cdot waiting - w_q \cdot queue + w_f \cdot flow$ ($w_w = w_q = w_f = 1$)	Récompense locale + composante globale de coordination (IQL); même formule de base
Interface de contrôle	TraCI [27]	TraCI [27]

2.2 Flux de trafic

Le profil de trafic adopté vise à reproduire les conditions asymétriques caractéristiques d'une heure de pointe matinale. Les probabilités d'arrivée ont été calibrées séparément pour chaque scénario.

2.2.1 Scénario 1 — Une intersection

Le tableau 2.2 présente les flux appliqués à l'intersection unique J1, composée des arêtes E0–E3.

TABLE 2.2 – *Flux de trafic — 1 intersection (probabilité d'arrivée/seconde)*

Origine	Destination	Probabilité
Sud (–E1)	Droite E2	0,01
Sud (–E1)	Tout droit –E0	0,06
Sud (–E1)	Gauche E3	0,02
Nord (–E2)	Droite –E0	0,01
Nord (–E2)	Tout droit E3	0,02
Nord (–E2)	Gauche E1	0,02
Est (–E3)	Droite E1	0,02
Est (–E3)	Tout droit E2	0,01
Est (–E3)	Gauche –E0	0,03
Ouest (E0)	Droite E3	0,01
Ouest (E0)	Tout droit E1	0,03
Ouest (E0)	Gauche E2	0,01

2.2.2 Scénario 2 — Deux intersections

Le réseau à deux intersections (J1 et J5) génère des flux sur six arêtes d'entrée : E0, E2, E3 pour J1, et E5, E6, E7 pour J5. Le trafic transite entre les deux carrefours via le tronçon interne E4/–E4. Le tableau 2.3 détaille les probabilités d'arrivée retenues.

TABLE 2.3 – *Flux de trafic — 2 intersections (probabilité d’arrivée/seconde)*

	Origine	Destination	Probabilité
J1	Ouest (E0)	Droite –E5	0,01
	Ouest (E0)	Tout droit –E7	0,01
	Ouest (E0)	Gauche –E6	0,03
	Nord (E2)	Droite –E6	0,02
	Nord (E2)	Tout droit –E5	0,01
	Nord (E2)	Gauche –E7	0,01
	Est (E3)	Droite –E7	0,01
	Est (E3)	Tout droit –E6	0,02
	Est (E3)	Gauche –E5	0,01
J5	Sud (E5)	Droite –E7	0,01
	Sud (E5)	Tout droit –E6	0,02
	Sud (E5)	Gauche –E2	0,03
	Ouest (E6)	Droite –E2	0,02
	Ouest (E6)	Tout droit –E5	0,01
	Ouest (E6)	Gauche –E7	0,01
	Nord (E7)	Droite –E5	0,01
	Nord (E7)	Tout droit –E2	0,02
	Nord (E7)	Gauche –E6	0,01

2.3 Paramètres de l’algorithme Q-Learning

Les hyperparamètres du Q-Learning conditionnent directement la vitesse de convergence et la qualité de la politique apprise. Leur choix résulte d’une combinaison de recommandations issues de la littérature et d’ajustements empiriques réalisés lors des premières phases d’expérimentation. Le tableau 2.4 récapitule les valeurs retenues et en justifie les choix.

TABLE 2.4 – Paramètres de l’algorithme Q-Learning et justification des choix

Paramètre	Valeur	Justification
Taux d’apprentissage α	0,1	Valeur faible favorisant une convergence stable; des valeurs élevées génèrent de l’instabilité [1].
Facteur de discount γ	0,95	Valeur proche de 1 incitant l’agent à maximiser les récompenses à long terme, cohérent avec un horizon de 3 600 s [1].
Epsilon initial ε	1,0	Exploration totale en début d’entraînement pour couvrir l’espace d’états.
Epsilon minimal	0,05	Maintien d’une exploration résiduelle de 5% pour éviter le sur-apprentissage [1].
Décroissance d’epsilon	0,999	Transition progressive de l’exploration vers l’exploitation sur 5 000 épisodes.
Edges surveillés NS	-E2, -E3	Voies d’entrée de l’axe Nord-Sud, couvrant les flux entrants représentatifs.
Edges surveillés EW	-E1, E0	Voies d’entrée de l’axe Est-Ouest, symétriques aux edges NS.
Phases des feux	PHASE_NS = 0, PHASE_EW = 2	Correspondance directe avec les phases SUMO définies dans le fichier réseau.
Sauvegarde	Toutes les 10 épisodes	Compromis entre fréquence de sauvegarde et coût I/O sur 5 000 épisodes.

2.4 Données extraites via TraCI

À chaque pas de décision, l’agent doit observer l’état du trafic pour mettre à jour sa politique. Ces observations sont extraites en temps réel par l’interface TraCI, qui constitue le pont entre la simulation SUMO et le code Python de l’agent [27]. Le tableau 2.5 présente les données collectées et leur rôle précis dans le processus d’apprentissage.

TABLE 2.5 – *Données extraites via TraCI et leur utilisation dans l'apprentissage*

Donnée	Méthode TraCI	Utilisation
Nombre de véhicules par voie	<code>getLastStepVehicle-</code> <code>Number (edge)</code>	Construction de l'état
Nombre de véhicules arrêtés	<code>getLastStepHalting-</code> <code>Number (edge)</code>	État principal de l'agent
Temps d'attente total	<code>getWaitingTime (edge)</code>	Calcul de la récompense
Phase actuelle des feux	<code>getPhase ("J1")</code>	Contrôle et décision

3 Diagramme d'Activité de l'Agent

Avant de détailler l'implémentation de chaque algorithme, il est utile de visualiser le comportement global de l'agent au fil d'un épisode. Le diagramme d'activité ci-après modélise ce comportement depuis l'observation initiale de l'état jusqu'à la mise à jour de la politique, en passant par la sélection de l'action, son exécution dans SUMO et le calcul de la récompense. Il met ainsi en évidence la nature cyclique et itérative de la boucle d'apprentissage par renforcement dans le contexte du contrôle des feux.

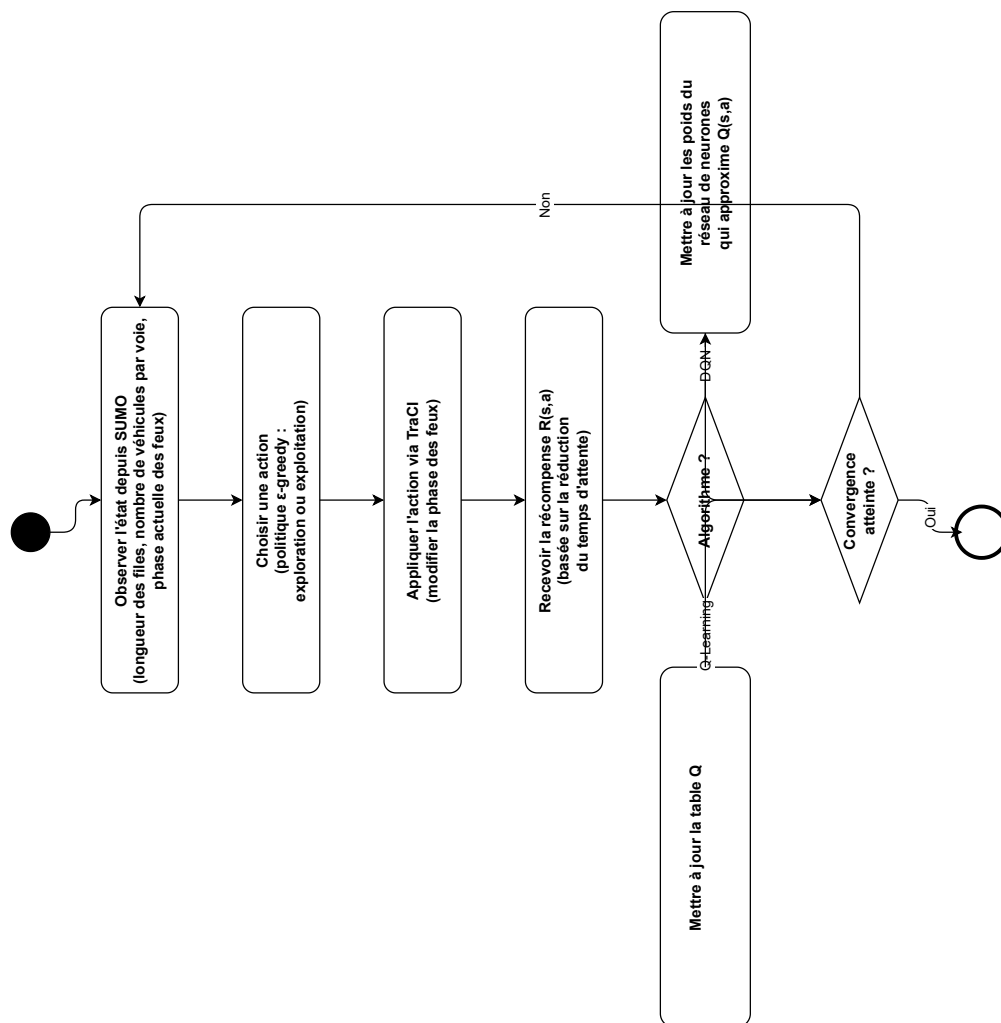


FIGURE 2.1 – *Diagramme d'activité de l'agent RL*

4 Q-Learning

Le Q-Learning constitue la première approche algorithmique mise en œuvre dans ce projet. Sa simplicité et ses garanties théoriques de convergence en font un choix naturel pour l'intersection unique, où l'espace d'états discrétisé reste de taille raisonnable. Cette section en expose les fondements théoriques, la règle de mise à jour et les modalités de son application au contrôle des feux.

Le Q-Learning, introduit par Watkins [3] et dont la convergence a été prouvée par Watkins et Dayan [4], est un algorithme de différence temporelle sans modèle et hors-politique (*off-policy*) : c'est-à-dire qu'il apprend la politique optimale indépendamment des actions réellement exécutées, ce qui lui permet de séparer la politique d'exploration de la politique cible. Il apprend directement la valeur optimale des paires état-action sans nécessiter de modèle explicite de l'environnement. Dans notre projet, il est appliqué au contrôle d'une intersection unique, où l'espace d'états réduit rend la méthode tabulaire parfaitement adaptée.

4.1 L'Équation d'Optimalité de Bellman

Le fondement théorique du Q-Learning est l'équation d'optimalité de Bellman [2]. Elle stipule que la valeur optimale d'une action a dans un état s est égale à la récompense immédiate espérée augmentée de la valeur actualisée de la meilleure action disponible dans l'état suivant :

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (2.4)$$

4.2 La Règle de Mise à Jour

Le Q-Learning met à jour les estimations de valeur sans attendre la fin d'un épisode, ce qui le rend particulièrement efficace dans des environnements à longue durée de simulation comme SUMO [4]. Après la transition (s, a, r, s') , la table Q est révisée comme suit :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot \delta, \quad \delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a) \quad (2.5)$$

où α est le taux d'apprentissage et δ l'erreur de différence temporelle (TD error) [1, 4].

4.3 Application au Contrôle des Feux

L'application concrète du Q-Learning au contrôle des feux nécessite de traduire le cadre théorique abstrait en mécanismes opérationnels compatibles avec l'interface TraCI de SUMO. L'état discret (ns_bin, ew_bin) est obtenu par discrétisation du nombre de véhicules arrêtés conformément aux BINS définis au tableau 2.4. Une contrainte de durée minimale de vert est appliquée : si cette durée n'est pas écoulée, l'action de commutation est ignorée. Les métriques sont enregistrées à chaque épisode pour l'analyse de convergence et la comparaison avec la baseline.

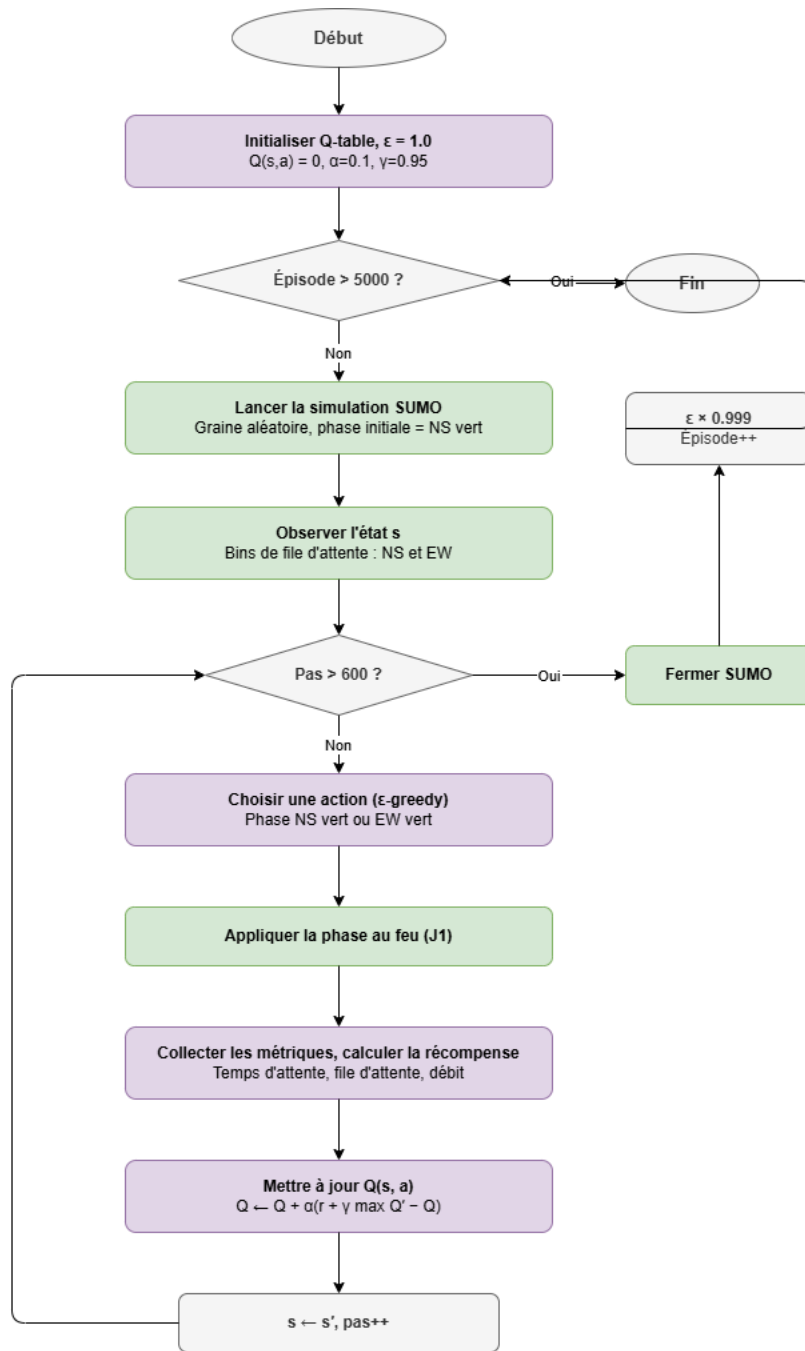


FIGURE 2.2 – Organigramme algorithmique du Q-Learning appliqué au contrôle des feux

5 Fonction d'Activation

La fonction d'activation est un élément clé de l'architecture neuronale du DQN, car elle détermine en grande partie la capacité du réseau à modéliser des relations non linéaires entre états et valeurs Q . Ce choix a des implications directes sur la vitesse de convergence et la stabilité de l'entraînement.

La fonction d'activation ReLU (Rectified Linear Unit), définie par $f(x) = \max(0, x)$,

est utilisée entre les couches cachées du réseau. Contrairement aux fonctions sigmoïde ou tangente hyperbolique, ReLU ne sature pas pour les valeurs positives, ce qui atténue le problème de disparition du gradient lors de la rétropropagation et accélère la convergence [5]. De plus, sa simplicité de calcul réduit le coût computationnel à chaque pas d'entraînement, ce qui est avantageux dans un contexte de simulation en temps réel avec SUMO.

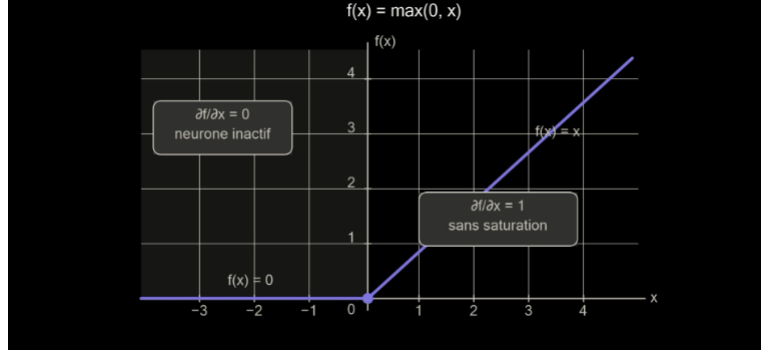


FIGURE 2.3 – Fonction d'activation ReLU : $f(x) = \max(0, x)$

6 Deep Q-Network (DQN)

Le Deep Q-Network représente l'extension naturelle du Q-Learning vers des espaces d'états plus complexes et des configurations multi-intersections. En substituant la table Q par un réseau de neurones, il acquiert une capacité de généralisation que le Q-Learning tabulaire ne peut pas offrir. Cette section détaille ses mécanismes de stabilisation, son architecture et sa procédure d'entraînement.

Le Deep Q-Network (DQN), proposé par Mnih et al. [5], résout la limitation de scalabilité du Q-Learning en approximant la fonction $Q(s, a)$ par un réseau de neurones $Q(s, a; \theta)$. Dans notre projet, le DQN est implémenté avec PyTorch pour le contrôle de deux intersections.

6.1 Mémoire de Relecture

À chaque pas, la transition (s, a, r, s') est stockée dans un tampon circulaire (*replay buffer*) [6]. Un mini-lot est tiré aléatoirement lors de l'entraînement, ce qui brise la corrélation temporelle entre expériences successives et permet à chaque transition de contribuer à plusieurs mises à jour. Dans le contexte du trafic, ce mécanisme permet de revisiter les épisodes de congestion marquée qui surviennent peu fréquemment mais sont essentiels à l'apprentissage.

6.2 Réseau Cible

Pour stabiliser l'entraînement, le DQN maintient deux réseaux [5] : le réseau principal $Q(s, a; \theta)$ mis à jour à chaque pas, et le réseau cible $Q(s, a; \theta^-)$ figé pendant C pas avant synchronisation. L'objectif est de minimiser la perte quadratique suivante :

$$\mathcal{L}(\theta) = \mathbb{E} [(y - Q(s, a; \theta))^2], \quad y = r + \gamma \max_{a'} Q(s', a'; \theta^-) \quad (2.6)$$

Cette stabilisation (2.6) prévient la boucle de rétroaction instable dans laquelle le réseau chasse une cible en perpétuel mouvement [5].

6.3 Architecture du Réseau de Neurons

L'architecture du réseau a été conçue pour rester suffisamment expressive tout en limitant le coût computationnel, compte tenu des contraintes de la simulation en temps réel. Le réseau prend en entrée le vecteur $[ns/10.0, ew/10.0]$, passe par deux couches cachées de 64 neurones avec activation ReLU (voir section 2.5), et produit deux valeurs Q (une par action). L'agent sélectionne l'action correspondant à la valeur Q maximale.

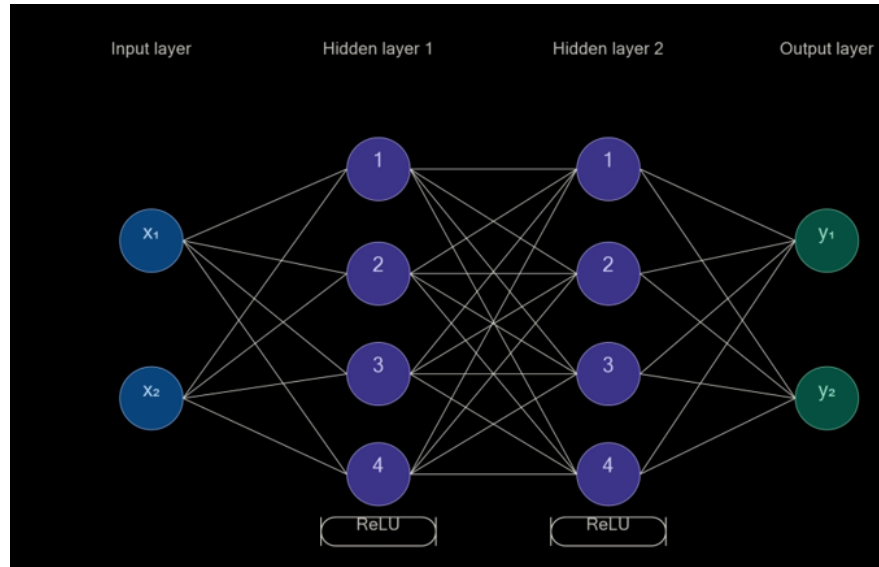


FIGURE 2.4 – Architecture du réseau de neurones du DQN pour le contrôle des feux sur une intersection unique

6.4 Procédure d'Entraînement

La procédure d'entraînement se déroule selon une boucle itérative où l'agent affine progressivement sa politique. Les étapes suivantes décrivent le cycle complet d'un pas de

décision jusqu'à la synchronisation périodique des réseaux.

1. **Initialisation** : le replay buffer est créé vide et les deux réseaux sont initialisés avec des poids identiques, assurant la cohérence des premières cibles.
2. **Observation** : l'agent récupère l'état initial $[ns/10.0, ew/10.0]$ depuis SUMO via TraCI.
3. **Sélection de l'action** : la politique ε -greedy est appliquée (voir section 2.1.3); ε décroît progressivement pour passer de l'exploration à l'exploitation.
4. **Exécution** : l'action est transmise à SUMO qui avance de 10 secondes et retourne la récompense (équation 2.1) et le nouvel état s' .
5. **Mémorisation** : le tuple (s, a, r, s') est stocké dans le replay buffer, permettant de revisiter les situations passées.
6. **Mise à jour** : un mini-lot est tiré aléatoirement, les cibles sont calculées via le réseau cible, et $\mathcal{L}(\theta)$ (équation 2.6) est minimisée par l'optimiseur Adam.
7. **Synchronisation** : tous les 50 pas, $\theta^- \leftarrow \theta$, stabilisant les cibles et prévenant les oscillations.
8. **Décrémentation de ε** : le paramètre d'exploration est réduit, puis le cycle reprend à l'étape 2.

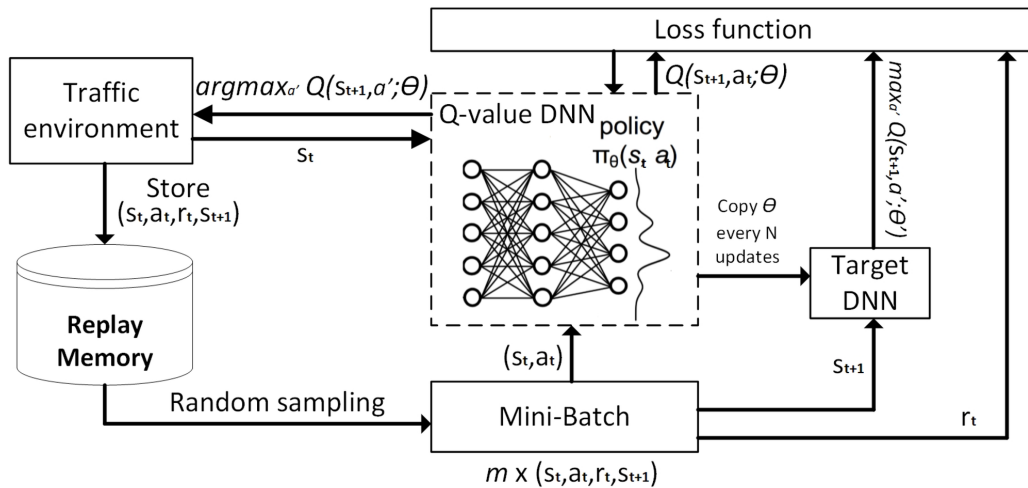


FIGURE 2.5 – Boucle d'entraînement du DQN avec mémoire de replay et réseau cible dans l'environnement de trafic SUMO

6.5 Extension Multi-Intersections

L'extension du contrôle à plusieurs intersections soulève la question de la coordination entre agents. Deux variantes architecturales ont été explorées afin d'évaluer leur scalabilité respective dans le cadre des approches Q-Learning (QL) et Deep Q-Learning (DQL).

6.5.1 Independent Q-Learning (IQL)

Le IQL propose une approche d'apprentissage par renforcement indépendant pour le contrôle de plusieurs intersections. Dans ce cadre, chaque intersection est considérée comme un agent autonome disposant de sa propre Q-table. Chaque agent apprend sa politique à partir d'une récompense locale, représentant les conditions de trafic à son intersection, enrichie par une composante globale de coordination afin de prendre en compte les interactions avec les intersections voisines. Cette approche permet un apprentissage décentralisé tout en maintenant une certaine coopération entre les agents.

6.5.2 Deep Q-Learning Partagé

Le DQL partagé propose un réseau de neurones unique partagé entre plusieurs intersections. Les intersections agissent indépendamment mais ne sont pas entraînées séparément : elles contribuent toutes à un ensemble commun de paramètres appris. Ce partage permet une meilleure généralisation et une plus grande efficacité d'apprentissage, notamment lorsque les intersections présentent des schémas de trafic similaires. Malgré cette architecture partagée, l'espace d'états et d'actions reste identique et simplifié, comme dans le cas mono-intersection.

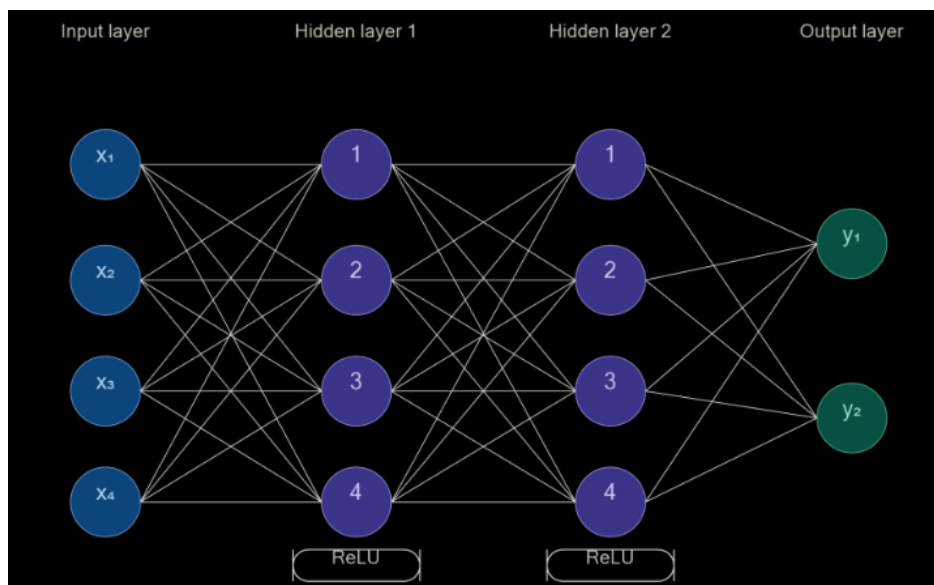


FIGURE 2.6 – Architecture du réseau de neurones du DQN pour le contrôle des feux sur deux intersections

7 Analyse Comparative : Q-Learning et DQN

Après avoir présenté chaque algorithme séparément, il convient d'en dresser une comparaison structurée afin d'identifier dans quels contextes chacun s'avère le plus pertinent. Le Q-Learning et le DQN partagent le même fondement théorique issu de l'équation de Bellman (2.4) [2], mais diffèrent architecturalement de manière significative. Ces différences ont des conséquences pratiques résumées dans le tableau 2.6.

TABLE 2.6 – Comparaison *Q-Learning* et *Deep Q-Network* dans le contexte du projet

Critère	Q-Learning	Deep Q-Network
Représentation	Table Q : (ns_bin, ew_bin)	Réseau PyTorch : $[ns/10, ew/10]$
Scalabilité	Limitée à l'état discret	Extensible aux espaces continus
Généralisation	Aucune	Interpolation vers états non vus
Stabilité	Non nécessaire	Replay buffer + réseau cible
Coût de calcul	Très faible	Modéré
Complexité	Simple et interprétable	Complexe, réglage fin requis
Convergence	Garantie (MDP fini) [4]	Empirique [5]

8 Baseline : Feux à Cycles Fixes

Le système de feux de circulation fixes repose sur un programme statique où les phases s'enchaînent de manière cyclique avec des durées prédéfinies, sans tenir compte de l'état du trafic. Dans notre cas, que ce soit pour une seule intersection ou pour deux intersections, le même cycle est appliqué, garantissant un fonctionnement simple et prévisible mais sans adaptation aux conditions réelles.

TABLE 2.7 – *Feux fixes : comparaison*

Élément	1 intersection	2 intersections
Contrôle	Statique	Statique identique
Phases	Cycle fixe 4 étapes	Cycle fixe identique
Durée	66 s	66 s
Adaptation	Aucune	Aucune
Coordination	Non	Non

Conclusion

Ce chapitre a posé l'intégralité du cadre de modélisation du projet. La formalisation du problème de contrôle des feux sous forme de Processus de Décision Markovien a permis d'ancrer notre approche dans un cadre théorique rigoureux. La configuration détaillée de l'environnement SUMO paramètres réseau, flux de trafic, hyperparamètres Q-Learning (avec justification des choix) et données TraCI fournit une base reproductible pour l'expérimentation. Le diagramme d'activité a illustré le comportement cyclique de l'agent, tandis que la présentation du Q-Learning et du DQN a mis en évidence leurs complémentarités : le premier, simple et à convergence garantie, est adapté à l'intersection unique ; le second, s'appuyant sur un réseau de neurones et des mécanismes de stabilisation, étend ces capacités aux configurations plus complexes. Le chapitre suivant présente l'évaluation expérimentale de ces approches dans des scénarios de simulation réalistes.

Chapitre 3

Évaluation

Introduction

Ce chapitre présente l'évaluation expérimentale des algorithmes de contrôle des feux développés au chapitre précédent. Les paramètres de simulation et les données d'apprentissage ayant été définis au chapitre 2, nous nous concentrons ici sur la mise en œuvre des scénarios de test, la définition des métriques d'évaluation et l'analyse comparative des résultats obtenus. L'objectif est de mesurer et de comparer les performances du Q-Learning, du Deep Q-Learning et d'une baseline à feux fixes dans deux configurations : une intersection unique et un réseau de deux intersections interconnectées. Cette double évaluation permet d'analyser à la fois les performances absolues de chaque approche et leur capacité à passer à l'échelle face à une complexité croissante.

1 Scénarios de simulation

Dans cette section, nous décrivons les différents scénarios utilisés pour évaluer les performances des approches proposées (Q-Learning et Deep Q-Learning) dans la gestion des feux de circulation. Les simulations sont réalisées à l'aide de SUMO, dont les paramètres de configuration ont été établis à la section 2.2, afin de reproduire des conditions de trafic réalistes et contrôlées.

1.1 Scénario 1 : Une seule intersection

Ce premier scénario correspond à un environnement simple composé d'une unique intersection à quatre branches. Il constitue le cas de base pour l'évaluation des algorithmes dans un environnement contrôlé et isolé.

- Le trafic est généré sur les axes Nord-Sud et Est-Ouest.
- L'objectif principal est de vérifier la capacité des agents à réduire :
 - le temps d'attente moyen,

- la congestion locale,
- la longueur des files d’attente.

Dans ce scénario épuré, le trafic suit une loi de probabilité connue, ce qui permet d’évaluer la performance fondamentale des algorithmes face à une stochasticité maîtrisée et non à des perturbations extérieures.

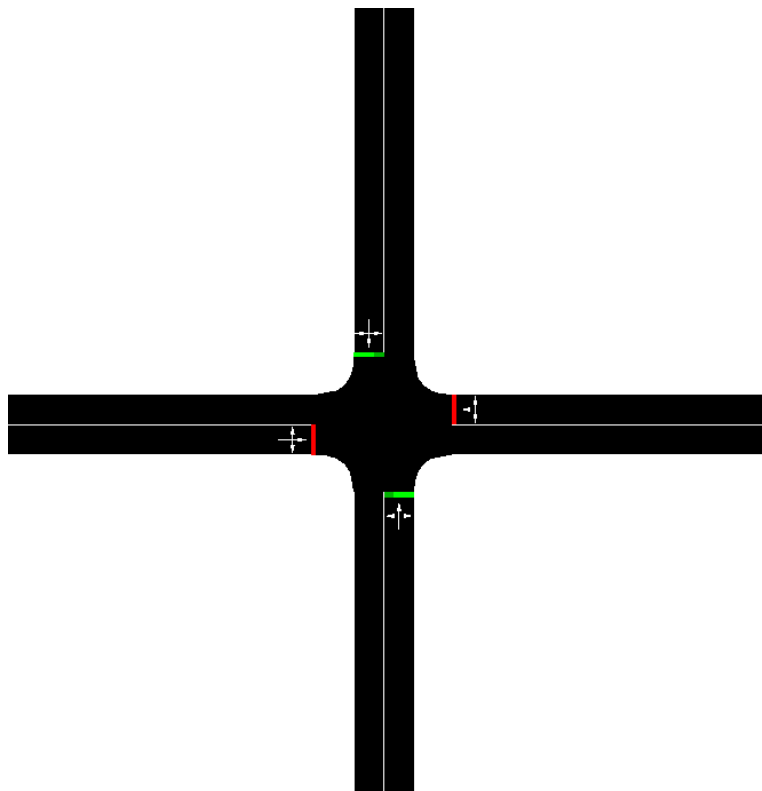


FIGURE 3.1 – *Vue SUMO de l’intersection unique à quatre branches (scénario 1)*

1.2 Scénario 2 : Deux intersections interconnectées

Le second scénario introduit un environnement plus complexe composé de deux intersections reliées par un axe commun. Chaque agent prend ses décisions localement, sans communication directe avec l’autre.

- Le trafic peut se propager d’une intersection à l’autre.
- Les flux de véhicules sont interdépendants, ce qui augmente la complexité de la décision.
- Chaque intersection est contrôlée indépendamment par un agent RL.

La congestion formée à une intersection peut se propager vers l’autre si les phases ne sont pas bien gérées, rendant le problème non-stationnaire du point de vue de chaque agent

individuel. L'objectif de ce scénario est d'évaluer la robustesse des modèles, leur capacité de généralisation, et leur supériorité par rapport à un contrôle à cycles fixes.

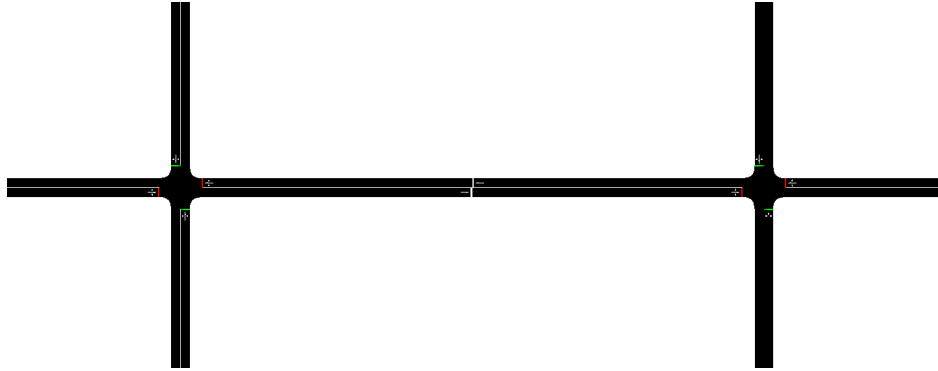


FIGURE 3.2 – *Vue SUMO du réseau à deux intersections interconnectées (scénario 2)*

1.3 Conditions communes aux deux scénarios

Afin d'assurer une comparaison équitable entre les méthodes (Q-Learning, DQN et feux à cycles fixes), les conditions suivantes sont maintenues constantes dans tous les scénarios :

- même durée de simulation,
- mêmes règles de priorité et contraintes de sécurité,
- même fonction de récompense $R = -\frac{1}{10} W - Q + F$ (équation 3.1) pour tous les agents.

1.4 Objectif des scénarios

Ces deux scénarios permettent de comparer :

- un environnement simple face à un environnement plus complexe,
- les performances des approches d'apprentissage par renforcement entre elles,
- et les systèmes RL face à un contrôle traditionnel basé sur des cycles fixes.

2 Métriques d'évaluation

L'évaluation des méthodes de contrôle des feux de circulation repose sur plusieurs métriques extraites directement de la simulation SUMO à l'aide de TraCI. Ces métriques permettent de comparer les performances des approches utilisées feux fixes, Q-Learning et Deep Q-Learning pour une et deux intersections.

2.1 Temps d'attente moyen

Le temps d'attente moyen est la métrique principale. Il est calculé à l'aide de `getWaitingTime()` (disponible via `traci.edge` ou `traci.lane` selon le cas) : le programme additionne les temps d'attente des véhicules sur les voies contrôlées, puis divise par le nombre total de pas de simulation (`MAX_STEPS`). Un temps d'attente plus faible indique que les véhicules franchissent plus rapidement l'intersection et que la stratégie de contrôle est plus efficace.

2.2 Longueur moyenne des files d'attente

La longueur moyenne des files d'attente est mesurée avec `getLastStepHaltingNumber()` (disponible via `traci.edge` ou `traci.lane`), qui donne le nombre de véhicules arrêtés sur les routes ou les voies surveillées. Cette valeur est accumulée à chaque pas de simulation, puis divisée par `MAX_STEPS`. Une file d'attente plus courte signifie que le contrôleur réduit efficacement la congestion autour de l'intersection.

2.3 Débit de véhicules

Le débit de véhicules (*throughput*) est calculé avec `getLastStepVehicleNumber()` (via `traci.edge`) sur les routes de sortie. Il indique le nombre de véhicules circulant sur les sorties de l'intersection au cours de la simulation. Plus cette valeur est élevée, plus le système permet à un grand nombre de véhicules de traverser l'intersection.

2.4 Fonction de récompense

La fonction de récompense joue un rôle central dans l'apprentissage, car elle traduit les objectifs métier en signal scalaire que l'agent cherche à maximiser. Cette sous-section en présente la formulation multi-objectif, la transformation en objectif scalaire et la normalisation retenue pour équilibrer les contributions des différents termes.

La fonction de récompense retenue est la suivante :

$$R = -\frac{1}{10} W - Q + F \quad (3.1)$$

où W désigne le temps d'attente cumulé, Q la longueur des files d'attente et F le débit de véhicules. Les sous-sections suivantes détaillent la construction de cette formule.

2.4.1 Formulation du problème multi-objectif

Le problème de contrôle des feux de circulation vise à :

- minimiser le temps d'attente : $\min W$
- minimiser la longueur des files : $\min Q$
- maximiser le débit : $\max F$

Ce qui définit le problème d'optimisation multi-objectif suivant :

$$\min(W, Q), \quad \max(F) \quad (3.2)$$

2.4.2 Transformation en objectif scalaire

L'apprentissage par renforcement requiert une fonction de récompense scalaire. On convertit donc les objectifs de minimisation en maximisation :

$$\min W \Rightarrow \max(-W) \quad (3.3)$$

$$\min Q \Rightarrow \max(-Q) \quad (3.4)$$

$$\max F \Rightarrow \max(F) \quad (3.5)$$

Ce qui donne la forme **naïve non-pondérée** :

$$R_{\text{naïf}} = -W - Q + F \quad (3.6)$$

2.4.3 Problème de déséquilibre des échelles

Des observations empiriques montrent que les ordres de grandeur des trois composantes sont très différents :

$$W \in [-1000, 0]$$

$$Q \in [0, 20]$$

$$F \in [0, 20]$$

Sans pondération, $W \gg Q, F$: la contribution typique de $-W$ atteint $[0, 1000]$, contre $[-20, 0]$ pour $-Q$ et $[0, 20]$ pour F . La récompense naïve (3.6) est alors **dominée par le temps d'attente**, conduisant l'agent à ignorer la longueur des files et le débit.

2.4.4 Normalisation par pondération

Pour équilibrer les contributions, on introduit un facteur d'échelle $\alpha = \frac{1}{10}$ sur W uniquement, ce qui donne l'équation (3.1) définie précédemment.

Après application de ce facteur, les plages de chaque terme deviennent comparables :

$$\frac{1}{10}W \in [0, 100], \quad Q \in [0, 20], \quad F \in [0, 20] \quad (3.7)$$

Les trois termes sont désormais du même ordre de grandeur : $|\frac{1}{10}W| \sim |Q| \sim |F|$.

Les poids retenus dans l'équation (3.1) sont donc :

- $w_{waiting} = -\frac{1}{10}$ (pénalisation du temps d'attente)
- $w_{queue} = -1$ (pénalisation de la longueur des files)
- $w_{flow} = +1$ (encouragement du débit)

Cette formulation pousse l'agent à apprendre une politique qui réduit le nombre de véhicules bloqués et leur temps d'attente, tout en augmentant le nombre de véhicules qui traversent l'intersection, avec des contributions comparables pour chacun des trois objectifs.

3 Résultats et analyse

Cette section présente et interprète les résultats expérimentaux obtenus après convergence des algorithmes sur les deux scénarios de simulation. Chaque approche est d'abord analysée à travers ses courbes d'entraînement, puis comparée aux autres méthodes via des tableaux synthétiques.

3.1 Résultats — 1 intersection

Dans cette section, nous comparons les performances de trois approches de contrôle des feux de circulation au niveau d'une seule intersection : Q-Learning (QL), Deep Q-Learning (DQN) et feux fixes (baseline). Les métriques utilisées sont le temps d'attente moyen, la longueur moyenne de la file et le flux moyen (débit).

3.1.1 Courbes d'entraînement du Q-Learning

Les figures 3.3 à 3.6 présentent les quatre courbes d'entraînement de l'agent Q-Learning.

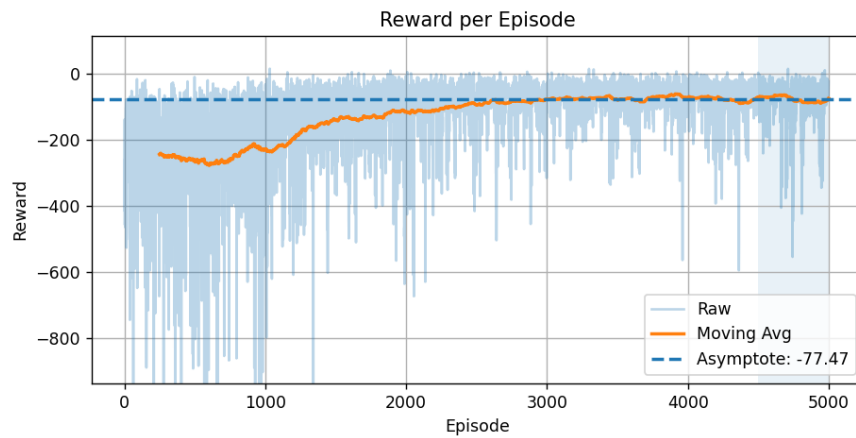


FIGURE 3.3 – *Q-Learning Récompense cumulée par épisode (asymptote : $-77,47$)*

La récompense présente une forte variance en phase d'exploration, puis converge progressivement vers une asymptote stable.

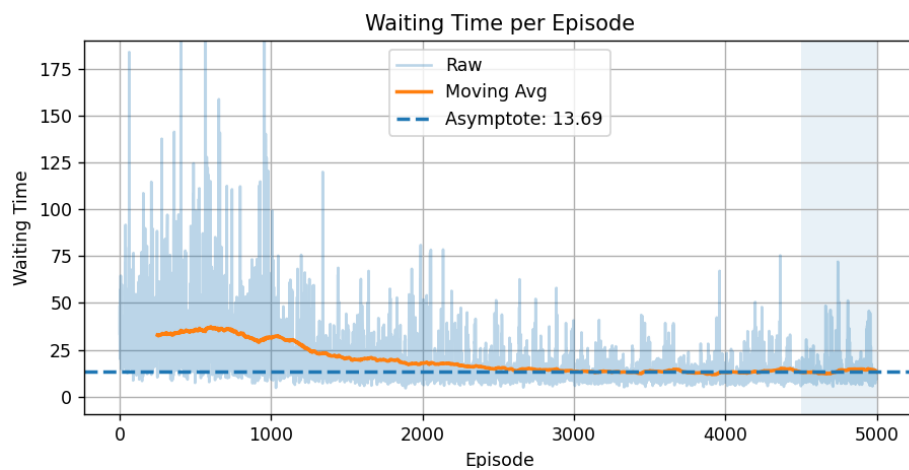


FIGURE 3.4 – *Q-Learning Temps d'attente moyen par épisode (asymptote : $13,69\text{ s}$)*

Le temps d'attente décroît régulièrement, traduisant l'amélioration progressive de la politique apprise.

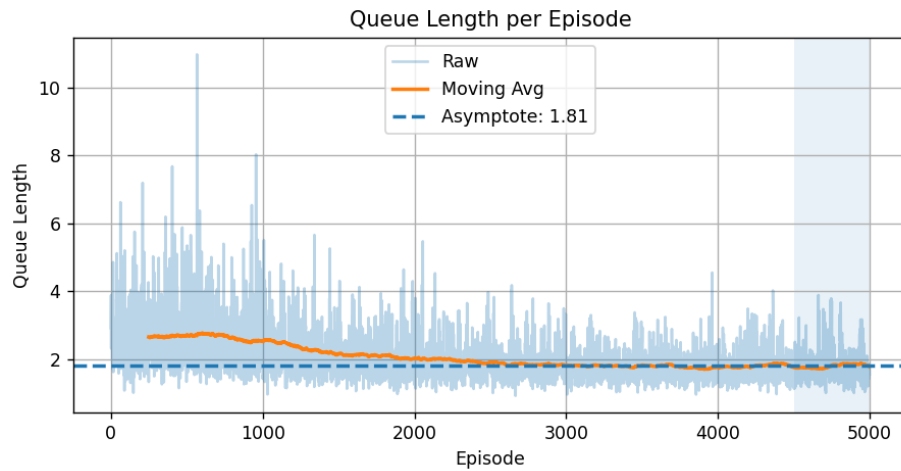


FIGURE 3.5 – *Q-Learning Longueur moyenne de file par épisode (asymptote : 1,81 véhicule)*

La longueur des files suit une trajectoire similaire, confirmant que l’agent apprend à arbitrer efficacement entre les deux axes.

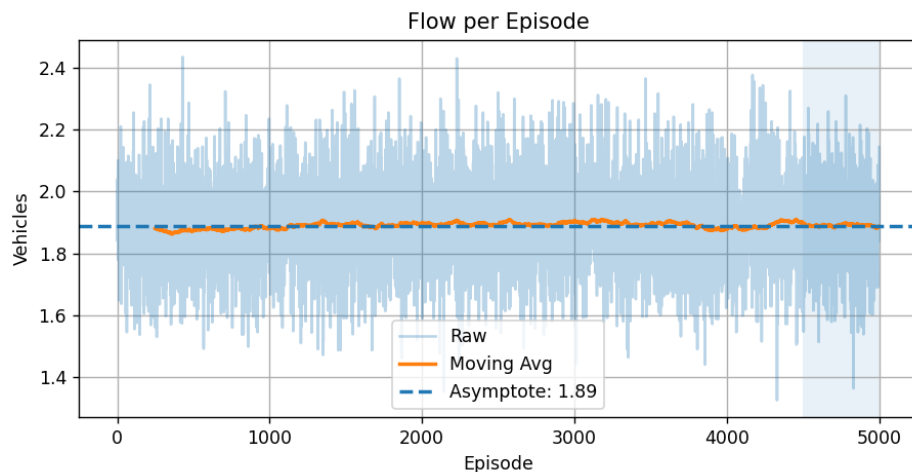


FIGURE 3.6 – *Q-Learning Flux moyen par épisode (asymptote : 1,89 véhicule/pas)*

Le débit reste stable, étant davantage contraint par le volume d’arrivée que par la stratégie de l’agent.

3.1.2 Courbes d’entraînement du DQN

Les figures 3.7 à 3.10 présentent les courbes d’entraînement de l’agent DQN.



FIGURE 3.7 – *DQN Récompense cumulée par épisode (asymptote : $-61,18$)*

La récompense atteint une asymptote supérieure à celle du Q-Learning, avec une convergence légèrement plus rapide.

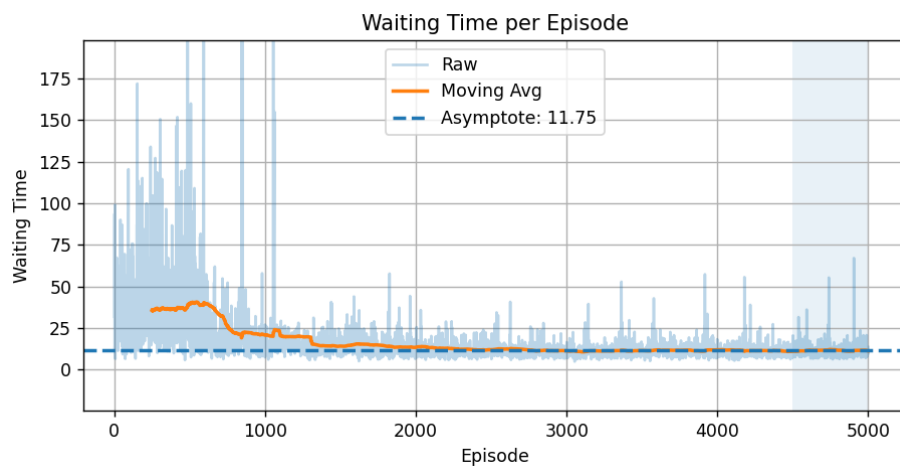


FIGURE 3.8 – *DQN Temps d'attente moyen par épisode (asymptote : $11,75$ s)*

Le temps d'attente converge vers une asymptote inférieure à celle du Q-Learning, confirmant la supériorité marginale du DQN sur cet indicateur.

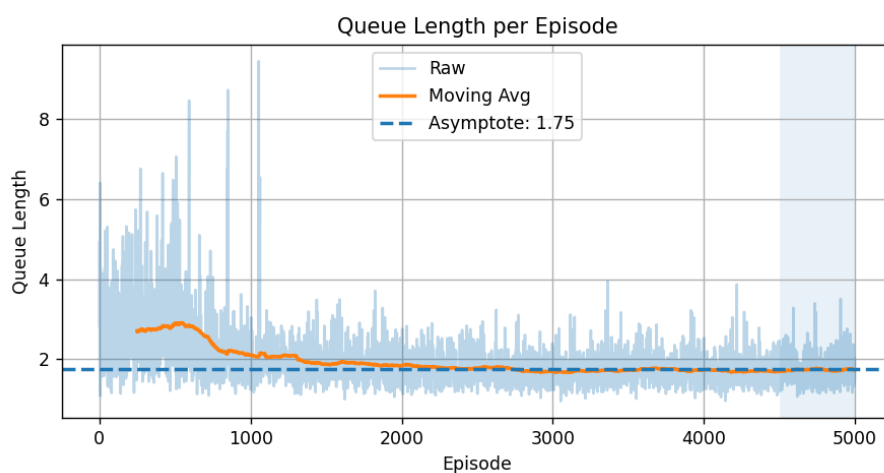


FIGURE 3.9 – *DQN Longueur moyenne de file par épisode (asymptote : 1,75 véhicule)*

Les files convergent avec des pics moins extrêmes, traduisant une exploration moins destructrice permise par la généralisation neuronale.

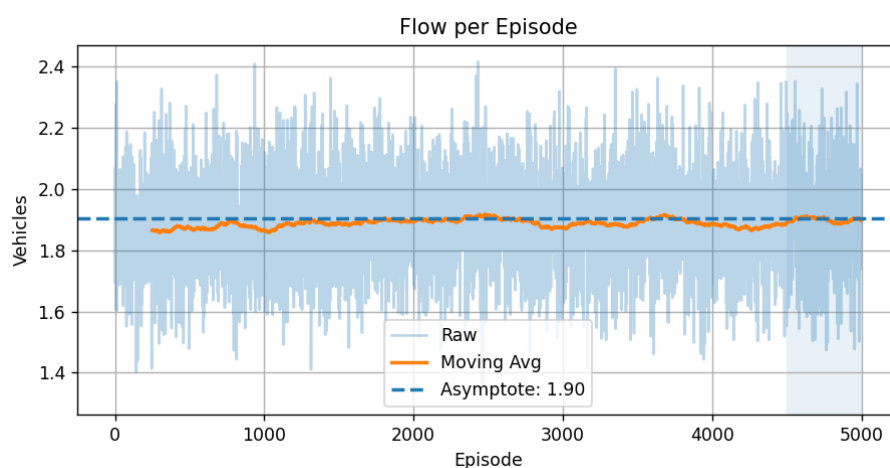


FIGURE 3.10 – *DQN Flux moyen par épisode (asymptote : 1,90 véhicule/pas)*

Comme pour le Q-Learning, le débit demeure stable dès les premiers épisodes.

3.1.3 Résultats comparatifs — 1 intersection

Le tableau 3.1 présente les résultats obtenus après convergence et évaluation sur 100 épisodes en régime stationnaire. Les pourcentages indiquent l'écart par rapport aux feux fixes (ligne de référence); la dernière ligne indique l'écart du DQN par rapport au Q-Learning.

TABLE 3.1 – Résultats comparatifs des trois approches — 1 intersection

Méthode	Temps d’attente (s)	Longueur de file (véh.)	Flux (véh./pas)
Feux fixes (réf.)	29,34	2,86	1,88
Q-Learning vs réf.	11,13 (−62,0%)	1,71 (−40,2%)	1,92 (+2,1%)
DQN vs réf.	10,87 (−63,0%)	1,70 (−40,5%)	1,92 (+2,1%)
DQN vs Q-Learning	−2,3%	−0,5%	0,0%

3.1.4 Conclusion — 1 intersection

Les résultats montrent que les approches basées sur l’apprentissage par renforcement permettent une réduction significative du temps d’attente, atteignant environ 62 à 63% par rapport aux feux fixes. De même, la longueur des files est réduite d’environ 40%. En revanche, l’amélioration du flux reste limitée ($\approx 2\%$), la capacité de l’intersection n’étant pas le facteur limitant dans ce scénario. La comparaison entre Q-Learning et DQN révèle que le DQN offre un gain marginal d’environ 2,3% sur le temps d’attente, tandis que les autres métriques restent pratiquement identiques.

3.2 Résultats — 2 intersections

Dans cette section, nous comparons les performances des trois mêmes approches dans un environnement plus complexe à deux intersections interconnectées. Les métriques évaluées sont identiques : temps d’attente moyen, longueur moyenne de la file et flux moyen.

3.2.1 Courbes d’entraînement du Q-Learning — 2 intersections

Chaque agent Q-Learning dispose d’une table indépendante et optimise sa propre intersection sans visibilité directe sur l’autre. Les figures 3.11 à 3.14 présentent les courbes d’entraînement obtenues. L’apprentissage indépendant reste efficace dans ce contexte couplé, bien que la convergence soit plus lente, ce qui reflète la non-stationnarité induite par le comportement simultané du second agent.

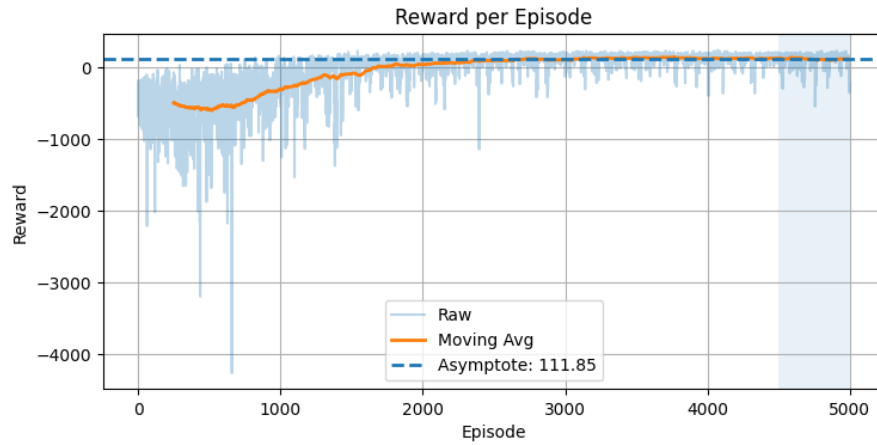


FIGURE 3.11 – *Q-Learning (2 intersections) Récompense cumulée par épisode*

La récompense présente une variance initiale élevée, caractéristique de la phase d’exploration dans un environnement non-stationnaire, avant de converger progressivement vers une asymptote stable.

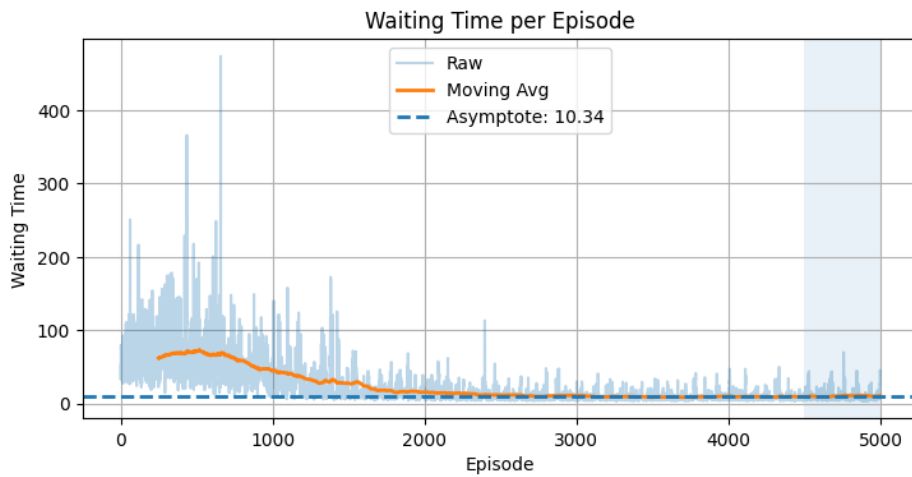


FIGURE 3.12 – *Q-Learning (2 intersections) Temps d’attente moyen par épisode*

Le temps d’attente décroît de façon régulière au fil des épisodes, traduisant l’amélioration progressive de la politique de chaque agent malgré les interdépendances de flux entre les deux carrefours.

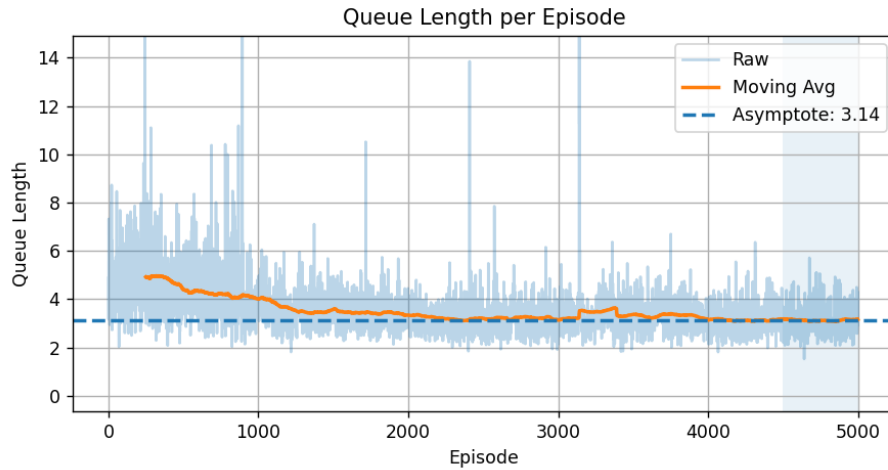


FIGURE 3.13 – *Q-Learning (2 intersections) Longueur moyenne de file par épisode*

La longueur des files suit une trajectoire de convergence similaire à celle du temps d'attente, confirmant que chaque agent apprend à arbitrer efficacement entre les phases malgré l'absence de communication avec l'agent voisin.

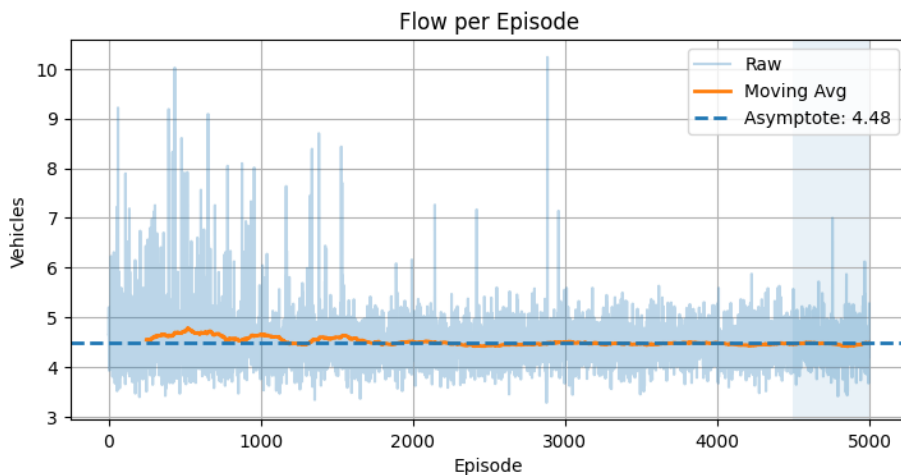


FIGURE 3.14 – *Q-Learning (2 intersections) Flux moyen par épisode*

Le débit reste globalement stable tout au long de l'entraînement, étant principalement conditionné par le volume de véhicules entrant dans le réseau plutôt que par la stratégie de l'agent.

3.2.2 Courbes d'entraînement du DQN — 2 intersections

Le DQN avec réseau partagé tire parti de la mutualisation des paramètres pour accélérer la convergence. Les figures 3.15 à 3.18 présentent les courbes d'entraînement correspondantes. La généralisation neuronale est particulièrement utile lorsque les états observés

varient davantage en raison des interdépendances de flux entre les deux carrefours.

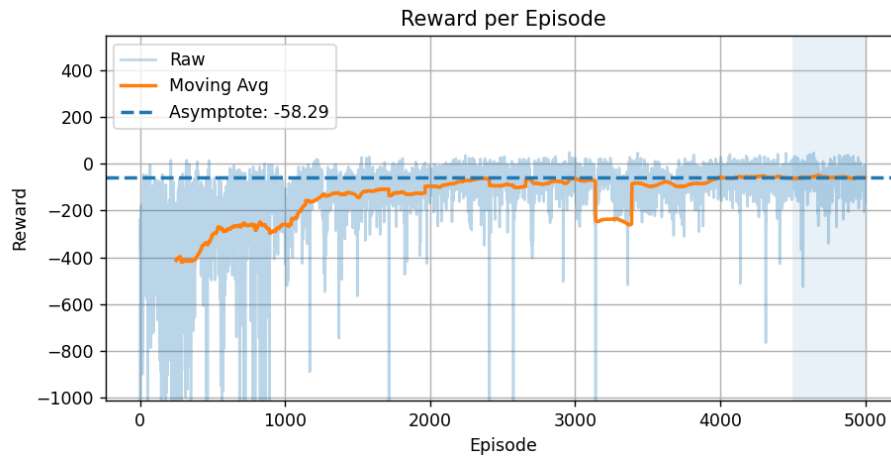


FIGURE 3.15 – *DQN (2 intersections) Récompense cumulée par épisode*

La récompense atteint une asymptote inférieure à celle du Q-Learning indépendant, ce qui reflète la difficulté accrue de la généralisation neuronale dans un environnement multi-agents non-stationnaire.

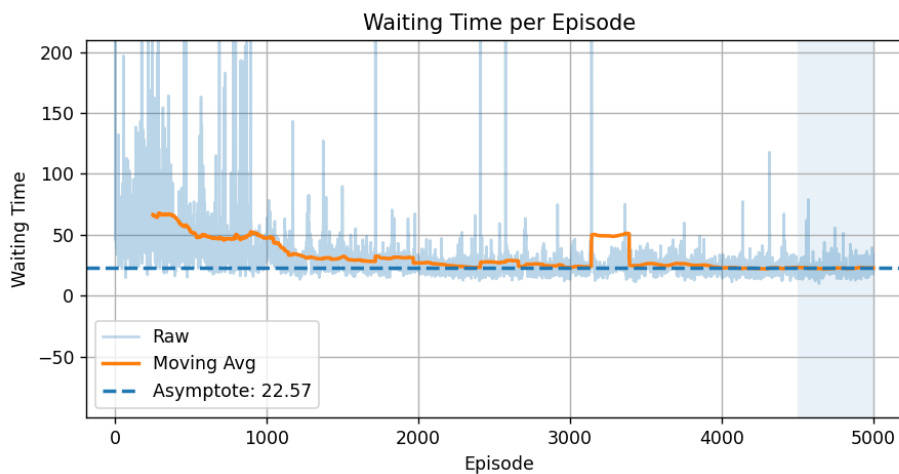


FIGURE 3.16 – *DQN (2 intersections) Temps d'attente moyen par épisode*

Le temps d'attente converge vers une valeur supérieure à celle obtenue par le Q-Learning, suggérant que la non-stationnarité multi-agents perturbe davantage la mise à jour du réseau de neurones que celle d'une table Q locale.

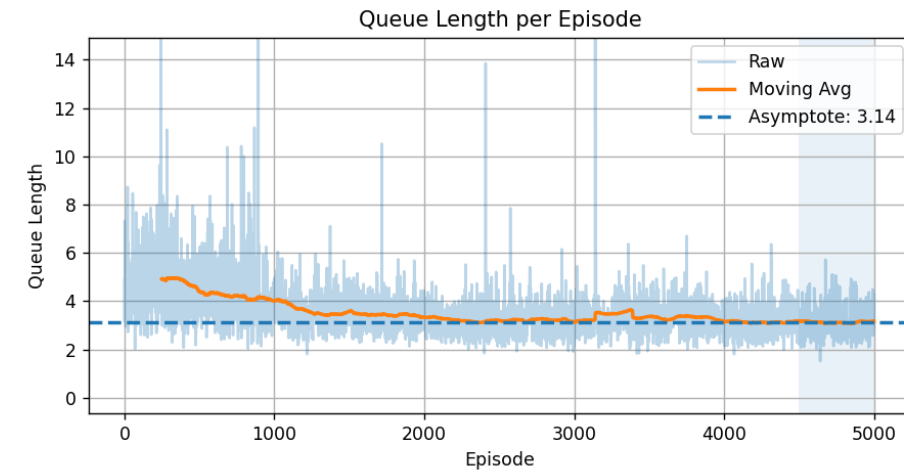


FIGURE 3.17 – *DQN (2 intersections) Longueur moyenne de file par épisode*

Les files d’attente se stabilisent à un niveau légèrement plus élevé que pour le Q-Learning, en cohérence avec la tendance observée sur le temps d’attente.

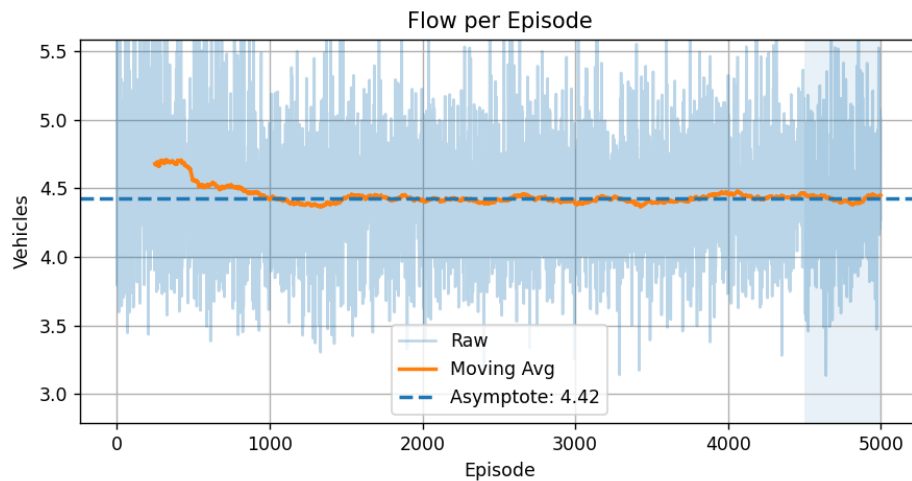


FIGURE 3.18 – *DQN (2 intersections) Flux moyen par épisode*

Comme pour le Q-Learning, le débit demeure stable dès les premiers épisodes et reste peu sensible aux variations de politique entre les deux agents.

3.2.3 Feux fixes — 2 intersections

Les feux fixes appliquent le même cycle sur les deux points sans coordination. Une vague libérée par la première intersection peut arriver en phase rouge à la seconde, aggravant les files de façon multiplicative.

3.2.4 Résultats comparatifs — 2 intersections

Le tableau 3.2 présente les résultats obtenus après convergence et évaluation sur 100 épisodes en régime stationnaire. Les pourcentages indiquent l'écart par rapport aux feux fixes (ligne de référence) ; la dernière ligne indique l'écart du DQN par rapport au Q-Learning.

TABLE 3.2 – *Résultats comparatifs des trois approches — 2 intersections*

Méthode	Temps d'attente (s)	Longueur de file (véh.)	Flux (véh./pas)
Feux fixes (réf.)	50,12	4,83	3,68
Q-Learning vs réf.	5,73 (−88,6%)	1,64 (−66,0%)	3,63 (−1,4%)
DQN vs réf.	9,87 (−80,3%)	2,38 (−50,7%)	3,61 (−1,9%)
DQN vs Q-Learning	+72,3%	+45,1%	−0,6%

3.2.5 Conclusion — 2 intersections

Les résultats montrent que les approches basées sur l'apprentissage par renforcement maintiennent de très bonnes performances dans un environnement plus complexe. Le Q-Learning se distingue particulièrement, avec une réduction du temps d'attente de près de 88,6% et une réduction des files de 66% par rapport aux feux fixes. Le DQN, bien qu'il reste très supérieur aux feux fixes ($\approx 80,3\%$ de réduction sur le temps d'attente), affiche dans ce scénario des résultats légèrement inférieurs à ceux du Q-Learning indépendant, ce qui s'explique par la plus grande difficulté de coordination implicite dans un environnement non-stationnaire à deux agents.

3.3 Comparaison globale

Le tableau 3.3 synthétise l'ensemble des résultats sur les deux scénarios, permettant une lecture transversale des performances de chaque approche en fonction de la complexité de l'environnement.

TABLE 3.3 – Comparaison globale des trois approches sur les deux scénarios

Scénario	Méthode	Temps (s)	File (véh.)	Flux (véh./pas)
1 intersection	Feux fixes	29,34	2,86	1,88
	Q-Learning	11,13 (−62%)	1,71 (−40%)	1,92 (+2%)
	DQN	10,87 (−63%)	1,70 (−41%)	1,92 (+2%)
2 intersections	Feux fixes	50,12	4,83	3,68
	Q-Learning	5,73 (−89%)	1,64 (−66%)	3,63 (−1%)
	DQN	9,87 (−80%)	2,38 (−51%)	3,61 (−2%)

Q-Learning vs DQN. Dans le scénario à une intersection, l’avantage du DQN est systématique mais marginal (−2,3% sur le temps d’attente). Dans le scénario à deux intersections, la situation s’inverse : le Q-Learning indépendant (IQL) surpasse nettement le DQN à réseau partagé (5,73 s contre 9,87 s), ce qui suggère que la non-stationnarité multi-agents perturbe davantage la généralisation neuronale que la table Q locale [5].

1 intersection vs 2 intersections. Le passage à deux intersections accentue considérablement la performance des approches RL par rapport aux feux fixes : le gain sur le temps d’attente passe de ∼62–63% (1 intersection) à ∼80–89% (2 intersections), et la réduction des files passe de ∼40% à ∼51–66%. Cela s’explique par le fait que les feux fixes subissent un effet multiplicatif de la congestion entre les deux carrefours, tandis que les agents RL s’y adaptent dynamiquement.

RL vs contrôle à temps fixe. Les approches RL surpassent largement le contrôle statique dans toutes les configurations. Le débit reste quasi stable (−1 à −2%), indiquant que le gain provient d’une meilleure gestion des phases et non d’une augmentation de la capacité physique. Ces résultats sont cohérents avec les travaux de Wei et al. [8, 9].

4 Discussion

Les résultats quantitatifs obtenus appellent une interprétation approfondie qui permet d’en comprendre les mécanismes sous-jacents, d’identifier les limites de l’approche et de proposer des pistes d’amélioration pour de futurs travaux.

4.1 Interprétation des résultats

Les résultats confirment la pertinence du RL pour la gestion intelligente des feux. La convergence régulière des moyennes mobiles vers des asymptotes stables atteste que les deux algorithmes sont correctement configurés et que leurs politiques sont stables, sans instabilité numérique ni oubli catastrophique.

4.2 Limites observées

En se limitant au nombre de véhicules arrêtés dans les deux directions, l’agent ignore des informations potentiellement utiles telles que la vitesse moyenne ou la durée de la phase courante, ce qui constitue un frein aux améliorations au-delà du seuil de convergence observé. L’extension au-delà de deux intersections reste par ailleurs un défi : la non-stationnarité s’accroît avec le nombre d’agents, et des architectures multi-agents centralisées seraient nécessaires à grande échelle. Enfin, les politiques ont été entraînées sur un profil de trafic fixe ; leur robustesse face à des variations de profil reste à évaluer dans des travaux futurs.

Conclusion

Ce chapitre a présenté l’évaluation expérimentale complète des algorithmes développés, en s’appuyant sur les paramètres de simulation établis au chapitre 2. Les résultats confirment que les approches RL surpassent systématiquement le contrôle à feux fixes dans toutes les configurations testées, avec des gains croissants à mesure que la complexité de l’environnement augmente. Le Q-Learning indépendant offre le meilleur compromis performance-complexité dans le scénario à deux intersections, tandis que le DQN affirme sa supériorité marginale en environnement simple. Ces résultats valident l’approche proposée et constituent une base solide pour des travaux futurs visant à étendre ces méthodes à des réseaux urbains plus réalistes.

Conclusion générale et perspectives

Ce projet avait pour ambition de démontrer que l'apprentissage par renforcement constitue une réponse viable et efficace au problème de la congestion urbaine, en proposant un système intelligent de contrôle des feux de signalisation capable d'apprendre et de s'adapter de manière autonome.

Les travaux menés ont permis d'atteindre l'ensemble des objectifs fixés. Sur le plan théorique, le problème de contrôle des feux a été rigoureusement formalisé sous forme de Processus de Décision Markovien, en définissant avec soin l'espace d'états, l'espace d'actions et la fonction de récompense multi-objectif. Sur le plan algorithmique, deux approches complémentaires ont été implémentées et comparées : le Q-Learning, dont la convergence est garantie dans un espace d'états fini, et le Deep Q-Network, qui généralise cette approche aux espaces continus grâce à l'approximation neuronale et aux mécanismes de stabilisation que sont la mémoire de relecture et le réseau cible.

L'évaluation expérimentale, conduite dans l'environnement de simulation SUMO sur deux scénarios de complexité croissante, a livré des enseignements contrastés. Dans le scénario à une intersection, les deux approches se sont révélées nettement supérieures au contrôle à feux fixes, avec un avantage marginal du DQN sur le Q-Learning, confirmant l'efficacité de la généralisation neuronale dans un environnement simple et stationnaire. Dans le scénario à deux intersections interconnectées, si les deux approches surpassent très largement les feux fixes, le DQN n'atteint pas le niveau de performance qu'on aurait pu escompter au regard de ses résultats en environnement mono-agent. Le Q-Learning indépendant s'avère paradoxalement plus efficace dans ce contexte, en raison de la nature non-stationnaire de l'environnement multi-agents : chaque intersection évoluant simultanément, les mises à jour du réseau de neurones poursuivent une cible en perpétuel mouvement, perturbant la convergence et dégradant la qualité de la politique apprise, là où la table Q locale, plus simple, demeure robuste.

Ces résultats constituent une base solide pour des travaux futurs, dont plusieurs axes peuvent être envisagés.

Sur le plan algorithmique, l'adoption de variantes plus avancées telles que le Double

DQN, le Dueling DQN ou le Proximal Policy Optimization pourrait améliorer la stabilité du DQN en environnement non-stationnaire. Des architectures multi-agents coopératives notamment MADDPG ou QMIX permettraient une coordination explicite entre les agents et pourraient combler l'écart de performance observé face au Q-Learning indépendant.

Sur le plan de la modélisation, l'enrichissement de l'espace d'états en y intégrant la vitesse moyenne des véhicules, la durée de la phase courante ou des données issues de capteurs hétérogènes permettrait à l'agent de prendre des décisions plus informées et d'anticiper les propagations de congestion entre carrefours.

Sur le plan de la scalabilité, l'extension à des réseaux urbains de grande taille appelle des architectures capables de coordonner un nombre élevé d'intersections tout en maintenant la robustesse face à la non-stationnarité croissante, qui constitue le principal obstacle identifié dans ce projet.

Sur le plan applicatif, le déploiement d'une interface web et mobile à destination des gestionnaires du trafic et des policiers routiers — correspondant à la Phase 6 de la méthodologie CRISP-DM identifiée comme perspective future tout au long de ce rapport constitue le prolongement naturel de ce projet vers un outil opérationnel à impact réel.

En définitive, ce projet confirme que l'apprentissage par renforcement offre un potentiel transformateur pour la gestion dynamique des feux de signalisation. Le Q-Learning indépendant s'impose comme l'approche la plus robuste dans des environnements multi-agents, tandis que le DQN, prometteur en configuration simple, requiert des adaptations architecturales supplémentaires pour exprimer pleinement son potentiel dans des réseaux interconnectés. Ces enseignements ouvrent la voie à des recherches ciblées vers des approches coopératives, capables de faire émerger une coordination globale du trafic à partir de décisions locales, et constituent ainsi un premier pas concret vers des villes plus fluides, moins polluées et plus résilientes.

Bibliographie

- [1] R. S. Sutton et A. G. Barto, *Reinforcement Learning : An Introduction*, 2^e éd. Cambridge, MA : MIT Press, 2018. [En ligne]. Disponible : <http://incompleteideas.net/book/the-book-2nd.html>. [Consulté le 15 avril 2025].
- [2] R. Bellman, *Dynamic Programming*. Princeton, NJ : Princeton University Press, 1957.
- [3] C. J. C. H. Watkins, *Learning from Delayed Rewards*, thèse de doctorat, University of Cambridge, Cambridge, Royaume-Uni, 1989.
- [4] C. J. C. H. Watkins et P. Dayan, « Q-learning, » *Machine Learning*, vol. 8, n° 3-4, pp. 279–292, mai 1992. [En ligne]. Disponible : <https://doi.org/10.1007/BF00992698>. [Consulté le 10 avril 2025].
- [5] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg et D. Hassabis, « Human-level control through deep reinforcement learning, » *Nature*, vol. 518, n° 7540, pp. 529–533, fév. 2015. [En ligne]. Disponible : <https://doi.org/10.1038/nature14236>. [Consulté le 12 avril 2025].
- [6] L.-J. Lin, « Self-improving reactive agents based on reinforcement learning, planning and teaching, » *Machine Learning*, vol. 8, n° 3-4, pp. 293–321, 1992. [En ligne]. Disponible : <https://doi.org/10.1007/BF00992699>. [Consulté le 10 avril 2025].
- [7] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot et N. de Freitas, « Dueling network architectures for deep reinforcement learning, » dans *Proceedings of the 33rd International Conference on Machine Learning (ICML 2016)*, New York, NY, pp. 1995–2003, juin 2016. [En ligne]. Disponible : <https://proceedings.mlr.press/v48/wangf16.html>. [Consulté le 18 avril 2025].
- [8] H. Wei, G. Zheng, H. Yao et Z. Li, « IntelliLight : A reinforcement learning approach for intelligent traffic light control, » dans *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2018)*,

- Londres, Royaume-Uni, pp. 2496–2505, août 2018. [En ligne]. Disponible : <https://doi.org/10.1145/3219819.3220096>. [Consulté le 20 avril 2025].
- [9] H. Wei, C. Chen, G. Zheng, K. Wu, V. Gayah, K. Xu et Z. Li, « PressLight : Learning max pressure control to coordinate traffic signals in arterial network, » dans *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2019)*, Anchorage, AK, pp. 1290–1300, août 2019. [En ligne]. Disponible : <https://doi.org/10.1145/3292500.3330949>. [Consulté le 20 avril 2025].
- [10] Organisation Mondiale de la Santé, *Global Status Report on Road Safety 2023*. Genève : OMS, 2023. [En ligne]. Disponible : <https://www.who.int/publications/i/item/9789240086517>. [Consulté le 5 mars 2025].
- [11] Agence Internationale de l'Énergie (AIE), *CO2 Emissions from Fuel Combustion : Overview*. Paris : AIE, 2024. [En ligne]. Disponible : <https://www.iea.org/reports/co2-emissions-in-2023>. [Consulté le 10 mars 2025].
- [12] INRIX, « 2023 Global Traffic Scorecard. » INRIX Research, 2024. [En ligne]. Disponible : <https://inrix.com/blog/2023-global-traffic-scorecard/>. [Consulté le 5 mars 2025].
- [13] R. P. Roess, E. S. Prassas et W. R. McShane, *Traffic Engineering*, 4^e éd. Upper Saddle River, NJ : Pearson Prentice Hall, 2011.
- [14] T. Cabannes, M. A. Vincentelli, A. Sundt, H. Signargout, E. Porter, V. Chiu, A. Bayen et J. M. Menezes, « The impact of GPS-enabled shortest path routing on mobility : a game theoretic approach, » *European Journal of Transport and Infrastructure Research*, vol. 18, n° 4, pp. 424–443, oct. 2018. [En ligne]. Disponible : <https://doi.org/10.18757/ejtir.2018.18.4.3253>. [Consulté le 8 mars 2025].
- [15] T. Tollazzi, *Alternative Types of Roundabouts : An Informational Guide*. Cham : Springer, 2015. [En ligne]. Disponible : <https://doi.org/10.1007/978-3-319-10906-7>. [Consulté le 8 mars 2025].
- [16] P. R. Lowrie, « SCATS, Sydney Co-ordinated Adaptive Traffic System : A traffic responsive method of controlling urban traffic, » Roads and Traffic Authority of New South Wales, Sydney, Australie, rapport technique, 1990.

- [17] P. B. Hunt, D. I. Robertson, R. D. Bretherton et M. C. Royle, « The SCOOT on-line traffic signal optimisation technique, » *Traffic Engineering and Control*, vol. 23, n° 4, pp. 190–192, avr. 1982.
- [18] C. P. Pappis et E. H. Mamdani, « A fuzzy logic controller for a traffic junction, » *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 7, n° 10, pp. 707–717, oct. 1977. [En ligne]. Disponible : <https://doi.org/10.1109/TSMC.1977.4309605>. [Consulté le 10 mars 2025].
- [19] M. B. Trabia, M. S. Kaseko et M. Ande, « A two-stage fuzzy logic controller for traffic signals, » *Transportation Research Part C : Emerging Technologies*, vol. 7, n° 6, pp. 353–367, déc. 1999. [En ligne]. Disponible : [https://doi.org/10.1016/S0968-090X\(99\)00026-8](https://doi.org/10.1016/S0968-090X(99)00026-8). [Consulté le 10 mars 2025].
- [20] J. J. Sanchez-Medina, M. J. Galan-Moreno et E. Rubio-Royo, « Traffic signal optimization in 'La Almozara' district in Saragossa under congestion conditions, using genetic algorithms, traffic microsimulation, and cluster computing, » *IEEE Transactions on Intelligent Transportation Systems*, vol. 11, n° 1, pp. 132–141, mars 2010. [En ligne]. Disponible : <https://doi.org/10.1109/TITS.2009.2034383>. [Consulté le 12 mars 2025].
- [21] F. Teklu, A. Sumalee et D. Watling, « A genetic algorithm approach for optimising traffic control signals considering routing, » *Computer-Aided Civil and Infrastructure Engineering*, vol. 22, n° 1, pp. 31–43, janv. 2007. [En ligne]. Disponible : <https://doi.org/10.1111/j.1467-8667.2006.00468.x>. [Consulté le 12 mars 2025].
- [22] J. Zhang, Y. Zheng et D. Qi, « Deep spatio-temporal residual networks for citywide crowd flows prediction, » dans *Proceedings of the 31st AAAI Conference on Artificial Intelligence (AAAI-17)*, San Francisco, CA, pp. 1655–1661, fév. 2017. [En ligne]. Disponible : <https://ojs.aaai.org/index.php/AAAI/article/view/10735>. [Consulté le 15 mars 2025].
- [23] Y. Lv, Y. Duan, W. Kang, Z. Li et F. Y. Wang, « Traffic flow prediction with big data : a deep learning approach, » *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, n° 2, pp. 865–873, avr. 2015. [En ligne]. Disponible : <https://doi.org/10.1109/TITS.2014.2345663>. [Consulté le 15 mars 2025].

- [24] K. H. Lim, K. Y. Chan, C. Saiago, M. Eisa et K. Morad, « Accurate vehicle counting in traffic monitoring using deep learning with camera sensor data, » dans *Proceedings of the 11th International Conference on Information Management and Engineering*, New York, NY, pp. 58–62, 2019.
- [25] ECSEL Joint Undertaking, « COMPASS4D : Cooperative Systems for Intelligent Road Safety. » Projet européen FP7, rapport final, 2016. [En ligne]. Disponible : <https://cordis.europa.eu/project/id/314552>. [Consulté le 18 mars 2025].
- [26] R. Wirth et J. Hipp, « CRISP-DM : Towards a standard process model for data mining, » dans *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, Manchester, Royaume-Uni, pp. 29–39, 2000.
- [27] A. Wegener, M. Piórkowski, M. Raya, H. Hellbrück, S. Fischer et J.-P. Hubaux, « TraCI : An interface for coupling road traffic and network simulators, » dans *Proceedings of the 11th Communications and Networking Simulation Symposium (CNS 2008)*, Ottawa, Canada, pp. 155–163, avr. 2008. [En ligne]. Disponible : <https://doi.org/10.1145/1400713.1400740>. [Consulté le 2 avril 2025].
- [28] Eclipse Foundation, « Eclipse SUMO — Simulation of Urban MObility. » [En ligne]. Disponible : <https://eclipse.dev/sumo/about/>. [Consulté le 1^{er} avril 2025].
- [29] Eclipse Foundation, « SUMO Documentation. » [En ligne]. Disponible : <https://sumo.dlr.de/docs/index.html>. [Consulté le 1^{er} avril 2025].
- [30] Eclipse Foundation, « SUMO at a Glance. » [En ligne]. Disponible : https://sumo.dlr.de/docs/SUMO_at_a_Glance.html. [Consulté le 1^{er} avril 2025].
- [31] D. Krajzewicz, G. Hertkorn, C. Rössel et P. Wagner, « SUMO (Simulation of Urban MObility) : An open-source traffic simulation, » dans *Proceedings of the 4th Middle East Symposium on Simulation and Modelling (MESM 2002)*, Sharjah, Émirats Arabes Unis, pp. 183–187, sept. 2002. [En ligne]. Disponible : <https://www.researchgate.net/publication/224793504>. [Consulté le 1^{er} avril 2025].

Résumé

Ce projet vise à développer un système intelligent de gestion du trafic urbain basé sur l'apprentissage par renforcement. L'objectif principal est de réduire la congestion aux carrefours en optimisant dynamiquement la durée des phases des feux de signalisation. Contrairement aux approches statiques, l'agent IA apprend en interagissant directement avec son environnement, sans jeu de données préexistant. Pour valider cette approche, nous avons utilisé le simulateur de trafic open-source SUMO. Deux algorithmes ont été implémentés et comparés : le Q-Learning tabulaire pour une intersection unique, et le Deep Q-Network (DQN) pour des configurations plus complexes. Les résultats expérimentaux montrent une réduction du temps d'attente moyen de 62 à 89 % par rapport aux feux à cycles fixes, selon le scénario. **Mots clés** : apprentissage par renforcement, feux de signalisation, gestion du trafic, Q-Learning, Deep Q-Network, SUMO

Abstract

This project aims to develop an intelligent urban traffic management system based on reinforcement learning. The main objective is to reduce congestion at intersections by dynamically optimizing traffic signal phase durations. Unlike static approaches, the AI agent learns by directly interacting with its environment, without any pre-existing dataset. To validate this approach, we used the open-source traffic simulator SUMO. Two algorithms were implemented and compared: tabular Q-Learning for a single intersection, and Deep Q-Network (DQN) for more complex configurations. Experimental results show a reduction in average waiting time of 62 to 89% compared to fixed-time traffic lights, depending on the scenario. **Keywords**: reinforcement learning, traffic lights, traffic management, Q-Learning, Deep Q-Network, SUMO

المخلص

يهدف هذا المشروع إلى تطوير نظام ذكي لإدارة حركة المرور الحضرية يعتمد على التعلم التعزيزي. الهدف الرئيسي هو تقليل الازدحام عند التقاطعات من خلال التحسين الديناميكي لمدة مراحل إشارات المرور. على عكس الأساليب الثابتة، يتعلم الوكيل الذكي بالتفاعل المباشر مع بيئته، دون الحاجة إلى مجموعة بيانات مسبقة. للتحقق من صحة هذه المقاربة، استخدمنا محاكي حركة المرور مفتوح المصدر SUMO. تم تنفيذ ومقارنة خوارزميتين: Q-Learning الجدولي لتقاطع واحد، و Deep Q-Network (DQN) للتكوينات الأكثر تعقيداً. تظهر النتائج التجريبية انخفاضاً في متوسط زمن الانتظار بنسبة تتراوح بين 62% و 89% مقارنة بإشارات المرور ذات الدورات الثابتة، حسب السيناريو. **الكلمات المفتاحية**: التعلم التعزيزي، إشارات المرور، إدارة حركة المرور، Q-Learning، شبكة Q العميقة، SUMO